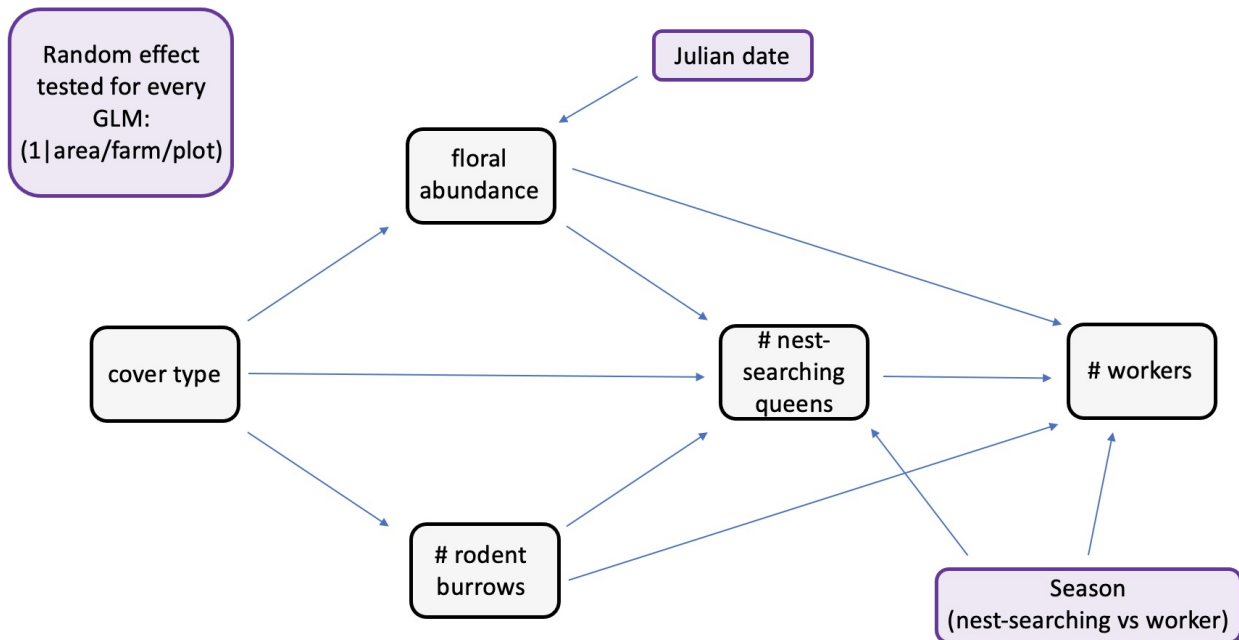


ch1_glm_clean

Predicted path diagram



Load and clean data

```
#load packages
library(tidyverse) #data manipulation, visualization etc. (dplyr, ggplot2, tidyr)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.1.4      v readr      2.1.5
## v forcats    1.0.0      v stringr   1.5.1
## v ggplot2    3.5.2      v tibble    3.3.0
## v lubridate  1.9.4      v tidyr     1.3.1
## v purrr      1.0.4
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(DHARMa)      #diagnostic tools for mixed regression models
```

```
## This is DHARMa 0.4.7. For overview type '?DHARMa'. For recent changes, type news(package = 'DHARMa')
```

```
library(ordinal)     #models for ordinal outcomes (e.g., cumulative link models)
```

```
##  
## Attaching package: 'ordinal'  
##  
## The following object is masked from 'package:dplyr':  
##  
##     slice
```

```
library(MASS)        #polr
```

```
##  
## Attaching package: 'MASS'  
##  
## The following object is masked from 'package:dplyr':  
##  
##     select
```

```
library(glmmTMB)      #fits generalized linear mixed models, including zero-inflated and overdispersed m  
library(car)          #tools for regression diagnostics and hypothesis tests
```

```
## Loading required package: carData  
##  
## Attaching package: 'car'  
##  
## The following object is masked from 'package:dplyr':  
##  
##     recode  
##  
## The following object is masked from 'package:purrr':  
##  
##     some
```

```
library(emmeans)     #calculates predicted results from models
```

```
## Welcome to emmeans.  
## Caution: You lose important information if you filter this package's results.  
## See '? untidy'
```

```
#read in data  
data <- read.csv("06_cleandata/combined_plot_data.csv")  
  
#change character columns to factors  
surveys <- data %>%  
  mutate(  
    
```

```

site_id = as.factor(site_id),
round = as.factor(round),
plot_id = as.factor(plot_id),
bee_date = as.Date(bee_date),
bee_season_type = as.factor(bee_season_type),
area = as.factor(area),
cover_type = as.factor(cover_type))

#scale continuous predictors
surveys$julian.date_scaled <- as.numeric(scale(surveys$julian.date))
surveys$bare_perc_scaled <- as.numeric(scale(surveys$bare_perc))
surveys$grass_perc_scaled <- as.numeric(scale(surveys$grass_perc))
surveys$forb_perc_scaled <- as.numeric(scale(surveys$forb_perc))
surveys$grass_forb_perc_scaled <- as.numeric(scale(surveys$grass_forb_perc))
surveys$grass_h_scaled <- as.numeric(scale(surveys$grass_h))
surveys$forb_h_scaled <- as.numeric(scale(surveys$forb_h))
surveys$floral_perc_scaled <- as.numeric(scale(surveys$floral_perc))
surveys$grass_forb_floral_perc_scaled <- as.numeric(scale(surveys$grass_forb_floral_perc))
surveys$bombus_floral_perc_scaled <- as.numeric(scale(surveys$bombus_floral_perc))
surveys$flood_perc_scaled <- as.numeric(scale(surveys$flood_perc))
surveys$dry_root_perc_scaled <- as.numeric(scale(surveys$dry_root_perc))
surveys$bryo_perc_scaled <- as.numeric(scale(surveys$bryo_perc))
surveys$bare_flood_dry_bryo_perc_scaled <- as.numeric(scale(surveys$bare_flood_dry_bryo_perc))
surveys$floral_richness_scaled <- as.numeric(scale(surveys$floral_richness))
surveys$bombus_floral_richness_scaled <- as.numeric(scale(surveys$bombus_floral_richness))
surveys$bombus_floral_abun_scaled <- as.numeric(scale(surveys$bombus_floral_abun))
surveys$bombus_floral_abun_nv_scaled <- as.numeric(scale(surveys$bombus_floral_abun_nv))
surveys$nest_searching_queens_scaled <- as.numeric(scale(surveys$nest_searching_queens))
surveys$workers_scaled <- as.numeric(scale(surveys$workers))
surveys$forage_vaco_scaled <- as.numeric(scale(surveys$forage_vaco))
surveys$num_burrows_scaled <- as.numeric(scale(surveys$num_burrows))
surveys$num_active_burrows_scaled <- as.numeric(scale(surveys$num_active_burrows))
surveys$num_inactive_burrows_scaled <- as.numeric(scale(surveys$num_inactive_burrows))
surveys$num_active_dm_scaled <- as.numeric(scale(surveys$num_active_dm))
surveys$num_dm_size_scaled <- as.numeric(scale(surveys$num_dm_size))
surveys$num_inactive_dm_scaled <- as.numeric(scale(surveys$num_inactive_dm))
surveys$prop_inactive_burrows_scaled <- as.numeric(scale(surveys$prop_inactive_burrows))
surveys$num_runway_burrows_scaled <- as.numeric(scale(surveys$num_runway_burrows))

#filter to only include field data
field_data <- surveys %>%
  filter(plot_id != "d")

```

Rodent burrows vs field management (%grass+forb)

Since the number of rodent burrows was count data, I knew I had to use either a poisson or negative binomial GLM family. I started with the simplest family (poisson) first. I didn't include julian.date as a covariate in this model since I assumed that the number of rodent burrows remains relatively consistent over the course of a season (hence why I only sampled each farm once).

Steps

- 1) Try different random effect structures in the poisson. Compare alternative random effect structures using maximum likelihood (ML) via AIC. I did not include plot as a random effect structure since there was only one observation per plot for rodent burrow data.

- best fit = 1|site_id -> **area not included**

```
burrows_full <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
  (1|area/site_id),
  family = poisson,
  data = field_data)

burrows_siteOnly <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
  (1|site_id),
  family = poisson,
  data = field_data)

burrows_areaOnly <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
  (1|area),
  family = poisson,
  data = field_data)

#compare models using maximum likelihood test
anova(burrows_full, burrows_siteOnly, burrows_areaOnly)

## Data: field_data
## Models:
## burrows_siteOnly: num_burrows ~ grass_forb_perc_scaled + (1 | site_id), zi=~0, disp=~1
## burrows_areaOnly: num_burrows ~ grass_forb_perc_scaled + (1 | area), zi=~0, disp=~1
## burrows_full: num_burrows ~ grass_forb_perc_scaled + (1 | area/site_id), zi=~0, disp=~1
##
##          Df    AIC    BIC   logLik deviance  Chisq Chi Df Pr(>Chisq)
## burrows_siteOnly  3 1670.5 1679.3  -832.25   1664.5
## burrows_areaOnly  3 3306.4 3315.2 -1650.18   3300.4    0.0      0      1
## burrows_full      4 1672.1 1683.9  -832.05   1664.1 1636.3      1 <2e-16
##
## burrows_siteOnly
## burrows_areaOnly
## burrows_full      ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

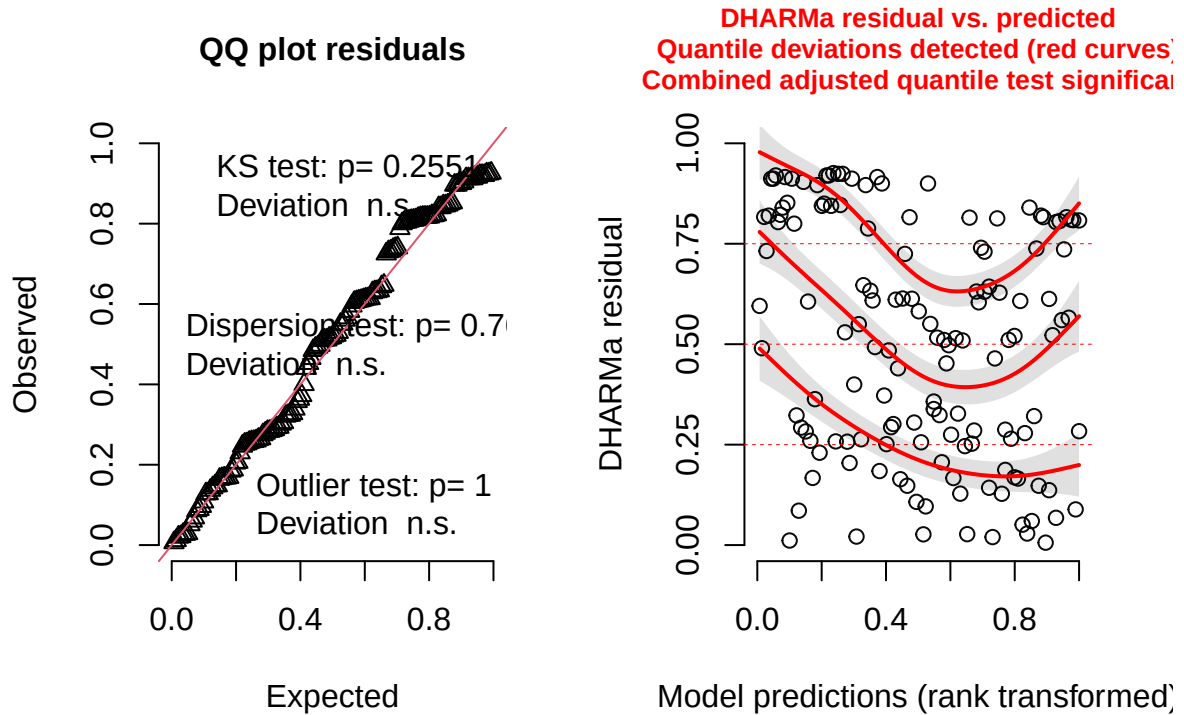
```
##results tells us the burrows_siteOnly is best model (lowest AIC)
```

- 2) Check diagnostics of the model using DHARMa package.

- Combined adjusted quantile test significant

```
#simulate residuals
residuals_burrows_siteOnly <- simulateResiduals(fittedModel = burrows_siteOnly, plot = TRUE)
```

DHARMA residual



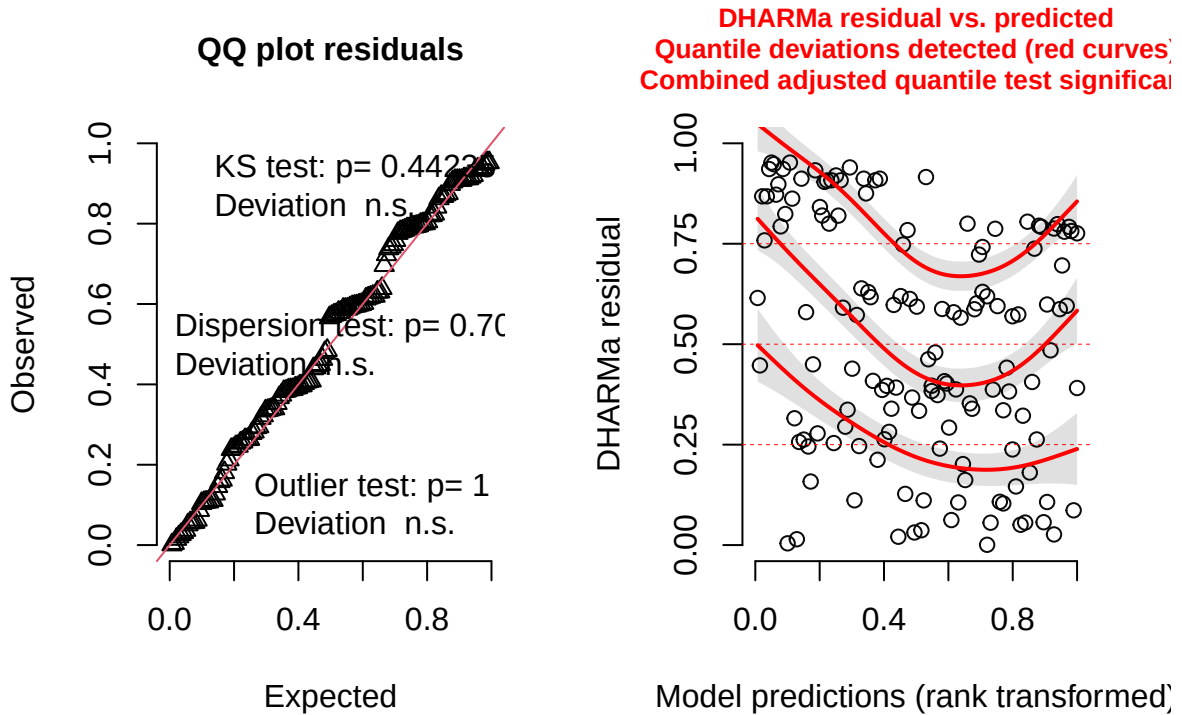
3) Try negative binomial instead and check diagnostics.

- Combined adjusted quantile test still significant.

```
burrows_siteOnly_negBin <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
  (1|site_id),
  family = nbinom2,
  data = field_data)

#simulate residuals
residuals_burrows_siteOnly_negBin <- simulateResiduals(fittedModel = burrows_siteOnly_negBin, plot = TRUE)
```

DHARMa residual



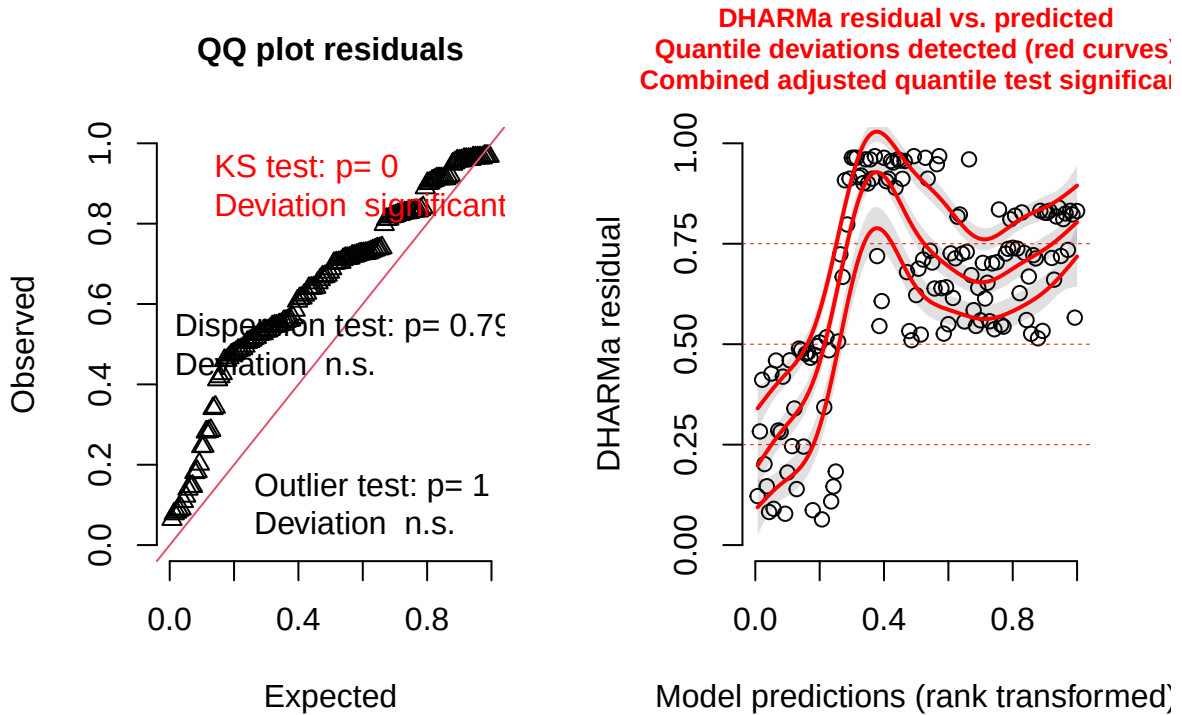
4) Try zero-inflated poisson and check diagnostics.

- combined adjusted quantile test still significant.

```
burrows_zi_pois <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
  (1|site_id),
  ziformula = ~ grass_forb_perc_scaled +
  (1|site_id),
  family = poisson,
  data = field_data)

#simulate residuals
residuals_burrows_zi <- simulateResiduals(fittedModel = burrows_zi_pois, plot = TRUE)
```

DHARMA residual



5) Try zero-inflated negative binomial and check diagnostics.

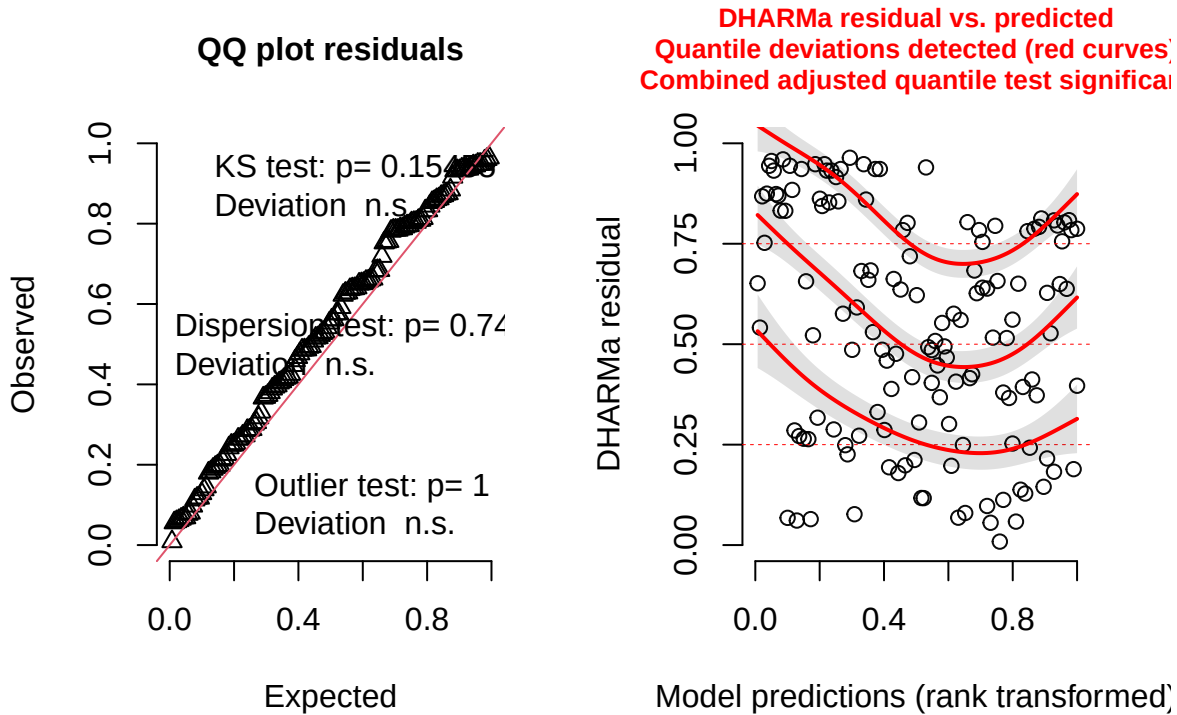
- combined adjusted quantile test still significant.

```
burrows_zi_nbin <- glmmTMB(num_burrows ~ grass_forb_perc_scaled +
  (1|site_id),
  ziformula = ~ grass_forb_perc_scaled +
  (1|site_id),
  family = nbinom2,
  data = field_data)

#simulate residuals
residuals_burrows_zi_nbin <- simulateResiduals(fittedModel = burrows_zi_nbin, plot = TRUE)
```

```
## Warning in newton(lsp = lsp, X = G$X, y = G$y, Eb = G$Eb, UrS = G$UrS, L = G$L,
## : Fitting terminated with step failure - check results carefully
```

DHARMa residual



#significant KS test and combined adjusted quantile test

- 6) Stick with poisson but try using categorical cover_type as predictor instead of % grass/forb cover and try different random effect structures.
- models using continuous percent grass/forb cover as a predictor showed poor fit and significant residual patterns, likely due to the small sample size ($n = 24$). Because sites were grouped into two distinct habitat types—grassy plots with high vegetation cover and bare plots with little or no cover—the variable was reclassified as a categorical factor (“grass” vs. “bare”). This simplified model structure improved residual diagnostics and provided a clearer ecological interpretation of differences in burrow abundance between habitat types.

```
burrows_cover_full <- glmmTMB(num_burrows ~ cover_type +
  (1|area/site_id),
  family = poisson,
  data = field_data)

burrows_cover_siteOnly <- glmmTMB(num_burrows ~ cover_type +
  (1|site_id),
  family = poisson,
  data = field_data)

burrows_cover_areaOnly <- glmmTMB(num_burrows ~ cover_type +
  (1|area),
  family = poisson,
```

```

data = field_data)

#compare models using maximum likelihood test
anova(burrows_cover_full, burrows_cover_siteOnly, burrows_cover_areaOnly)

## Data: field_data
## Models:
## burrows_cover_siteOnly: num_burrows ~ cover_type + (1 | site_id), zi=~0, disp=~1
## burrows_cover_areaOnly: num_burrows ~ cover_type + (1 | area), zi=~0, disp=~1
## burrows_cover_full: num_burrows ~ cover_type + (1 | area/site_id), zi=~0, disp=~1
##
##          Df      AIC      BIC    logLik deviance  Chisq Chi Df
## burrows_cover_siteOnly  3 1668.2 1677.0  -831.11   1662.2
## burrows_cover_areaOnly  3 2664.6 2673.4 -1329.28   2658.6    0.00      0
## burrows_cover_full     4 1667.7 1679.5  -829.87   1659.7  998.81      1
##
##          Pr(>Chisq)
## burrows_cover_siteOnly
## burrows_cover_areaOnly      1
## burrows_cover_full        <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

##full model is best (lowest AIC)

```

7. Check diagnostics

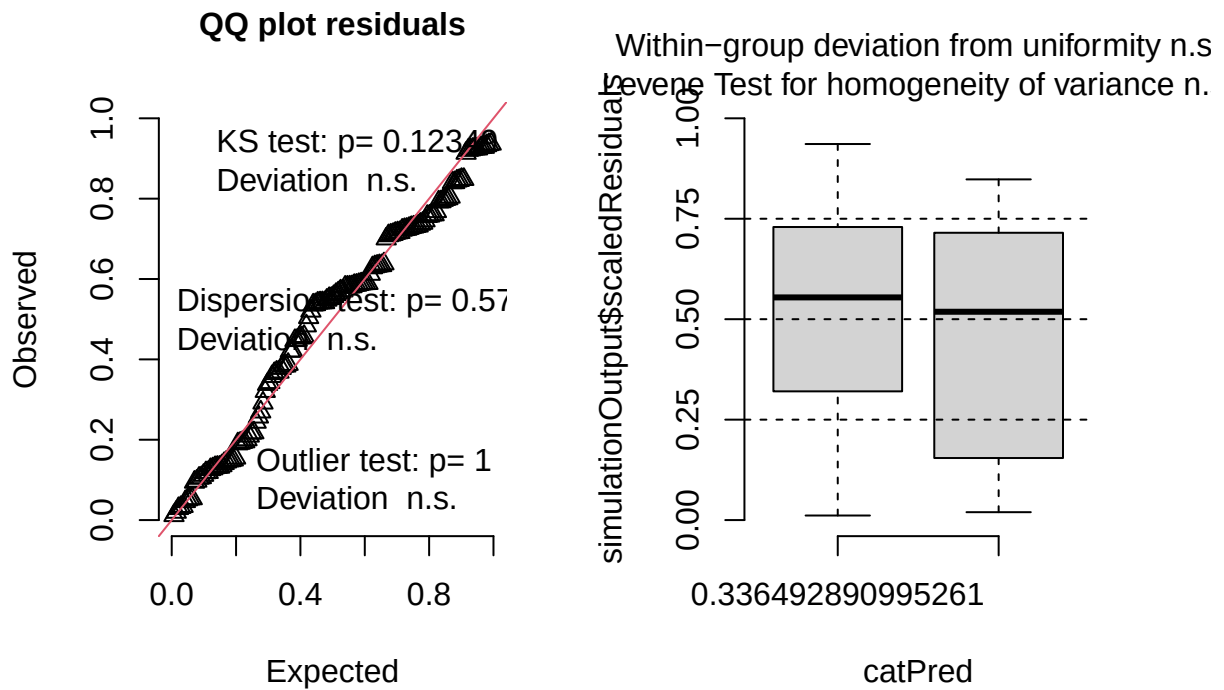
- significant within-group deviations from uniformity

```

#simulate residuals
residuals_burrows_cover_full <- simulateResiduals(fittedModel = burrows_cover_full, plot = TRUE)

```

DHARMA residual

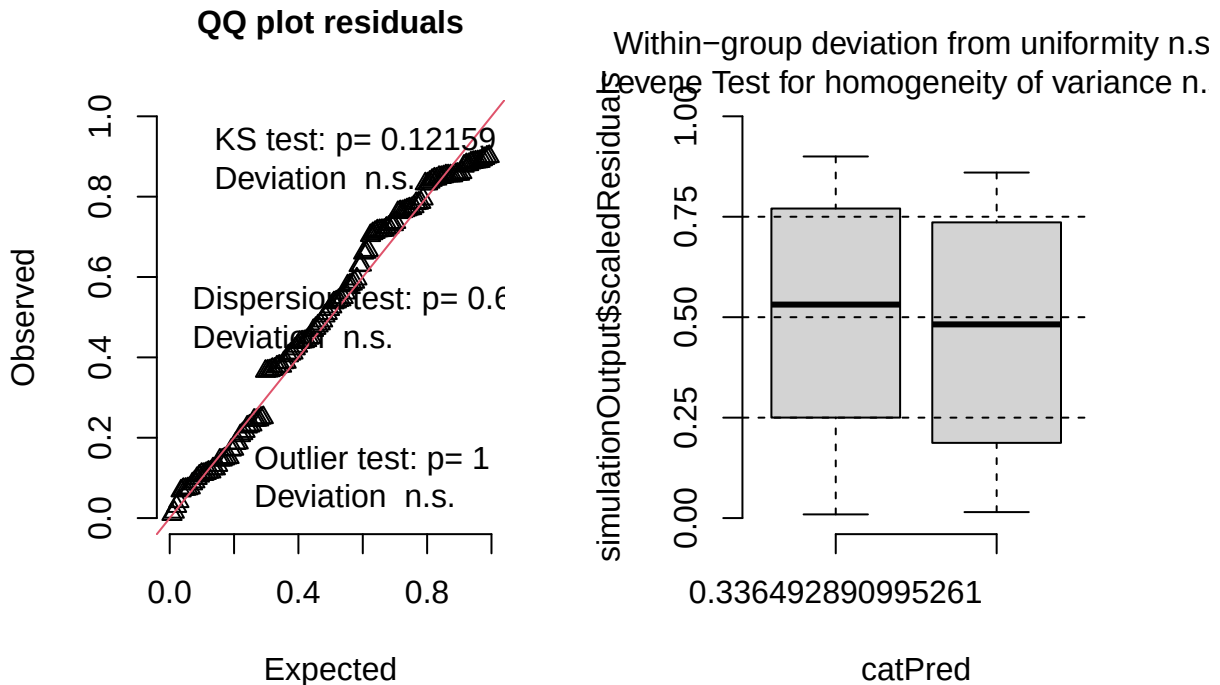


8. Try simpler random effect structure (site-only) and re-check diagnostics

- No significant problems detected.

```
#simulate residuals
residuals_burrows_cover_siteOnly <- simulateResiduals(fittedModel = burrows_cover_siteOnly, plot = TRUE)
```

DHARMA residual



9. Plot results

```
summary(burrows_cover_siteOnly)
```

```
## Family: poisson ( log )
## Formula:      num_burrows ~ cover_type + (1 | site_id)
## Data: field_data
##
##           AIC      BIC    logLik -2*log(L)  df.resid
##      1668.2    1677.0    -831.1    1662.2      137
##
## Random effects:
##
## Conditional model:
##   Groups  Name      Variance Std.Dev.
##   site_id (Intercept) 3.825    1.956
## Number of obs: 140, groups:  site_id, 12
##
## Conditional model:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    0.6365    0.8271   0.770   0.4415
## cover_typegrass 1.9894    1.1510   1.728   0.0839 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Plots

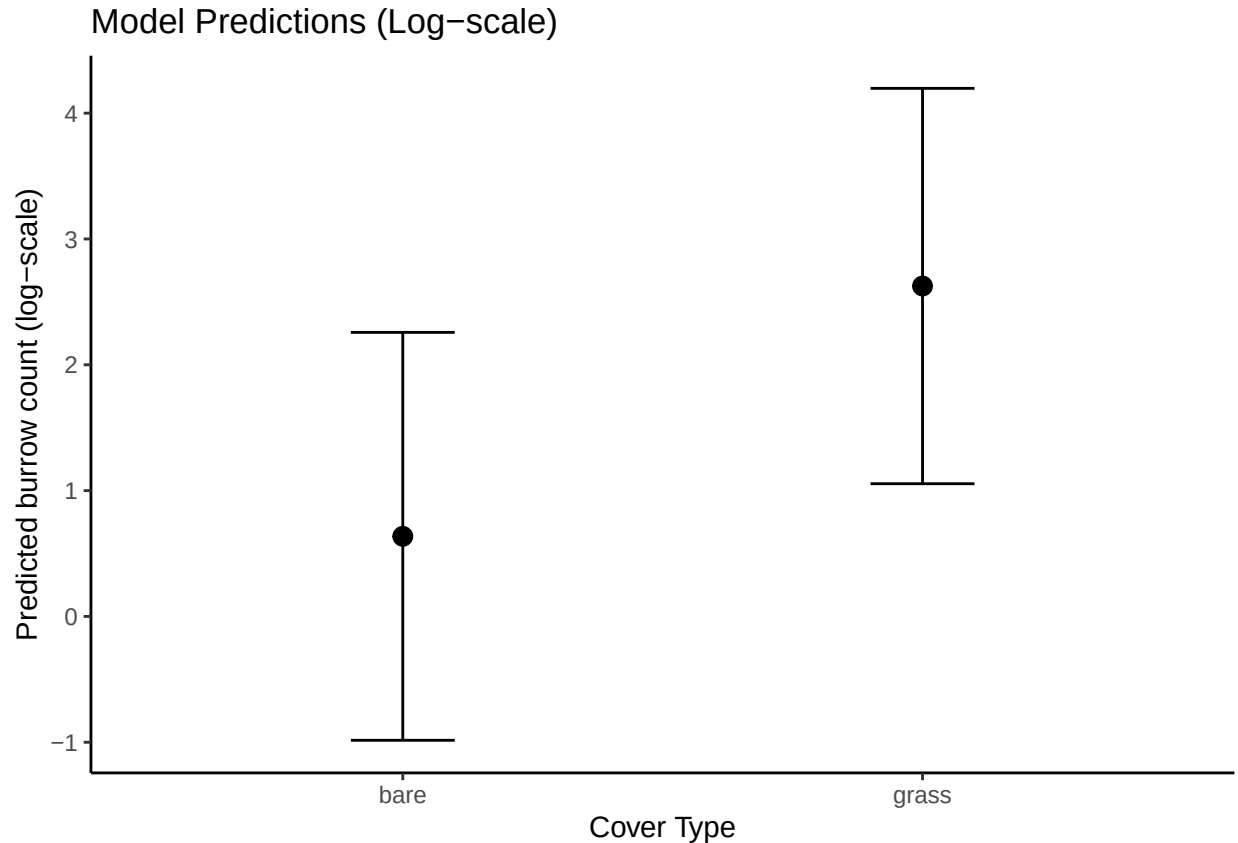
Number of rodent burrows vs cover type

```
#create new data grid
rodent_data_cover<- expand.grid(
  grass_forb_perc_scaled = mean(field_data$grass_forb_perc_scaled, na.rm = TRUE),
  cover_type = unique(field_data$cover_type),
  julian.date_scaled = mean(field_data$julian.date_scaled, na.rm = TRUE)
)

# Get predicted probabilities and standard errors
pred_rodent_cover <- predict(burrows_cover_siteOnly,
                             newdata = rodent_data_cover,
                             se.fit = TRUE,
                             re.form = NA)

# Combine predictions with data frame
pred_rodent_cover_df <- cbind(rodent_data_cover,
                              fit = pred_rodent_cover$fit,
                              se = pred_rodent_cover$se.fit)

ggplot(pred_rodent_cover_df, aes(x = cover_type, y = fit)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = fit - 1.96*se, ymax = fit + 1.96*se), width = 0.2, position = position_dodge)
labs(title = "Model Predictions (Log-scale)",
     x = "Cover Type",
     y = "Predicted burrow count (log-scale)") +
  theme_classic()
```



Findings

Neither grass + forb % nor julian date had significant effects on rodent burrow abundance. However, cover type showed a marginally insignificant trend ($p=0.0839$), where grassy sites had a higher abundance of burrows.

Floral abundance vs field management (%grass+forb)

Use cumulative link model aka ordinal logistic regression because bombus-relevant floral abundance has:

- discrete levels (0-5 scale) representing binned counts of blooms
- ordered categories (higher numbers represent more blooms)
- unequal intervals between categories

Steps

- 1) Tried running CLM with different random effect structures (full = 1|area/site/plot) then compared AIC values to determine best model.
 - determined that model without random effect had best fit

```

floral_full <- clmm(as.factor(bombus_floral_abun) ~ cover_type +
                    julian.date_scaled +
                    (1|area/site_id/plot_id),
                    link = "logit",
                    data = field_data)

floral_noArea <- clmm(as.factor(bombus_floral_abun) ~ cover_type +
                      julian.date_scaled +
                      (1|site_id/plot_id),
                      link = "logit",
                      data = field_data)

#floral_plotOnly <- clmm(as.factor(bombus_floral_abun) ~ cover_type +
#                          julian.date_scaled +
#                          (1|plot_id),
#                          link = "logit",
#                          data = field_data)
##can't use since there are only 2 levels

floral_siteOnly <- clmm(as.factor(bombus_floral_abun) ~ cover_type +
                        julian.date_scaled +
                        (1|site_id),
                        link = "logit",
                        data = field_data)

floral_AreaPlot <- clmm(as.factor(bombus_floral_abun) ~ cover_type +
                        julian.date_scaled +
                        (1|area/plot_id),
                        link = "logit",
                        data = field_data)

floral_noRE <- polr(as.factor(bombus_floral_abun) ~ cover_type +
                    julian.date_scaled,
                    method = "logistic",
                    data = field_data)

#compare models AIC
AIC(floral_full, floral_noArea, floral_siteOnly, floral_AreaPlot, floral_noRE)

```

```

##           df      AIC
## floral_full    9 441.1731
## floral_noArea  8 439.1731
## floral_siteOnly 7 437.1731
## floral_AreaPlot 8 439.1731
## floral_noRE    6 435.1731

```

```
##shows that model without random effect has best fit
```

2. Run final model and look at output

```
summary(floral_noRE)
```

```

##
## Re-fitting to get Hessian

```

```
## Call:
## polr(formula = as.factor(bombus_floral_abun) ~ cover_type + julian.date_scaled,
##       data = field_data, method = "logistic")
##
## Coefficients:
##               Value Std. Error  t value
## cover_typegrass    0.02647    0.3041  0.08702
## julian.date_scaled -0.23119    0.1527 -1.51417
##
## Intercepts:
##      Value  Std. Error t value
## 0|1 -1.3986  0.2610   -5.3585
## 1|2 -0.9291  0.2414   -3.8493
## 2|3 -0.1853  0.2272   -0.8153
## 3|4  0.6192  0.2332    2.6559
##
## Residual Deviance: 423.1731
## AIC: 435.1731
```

Plots

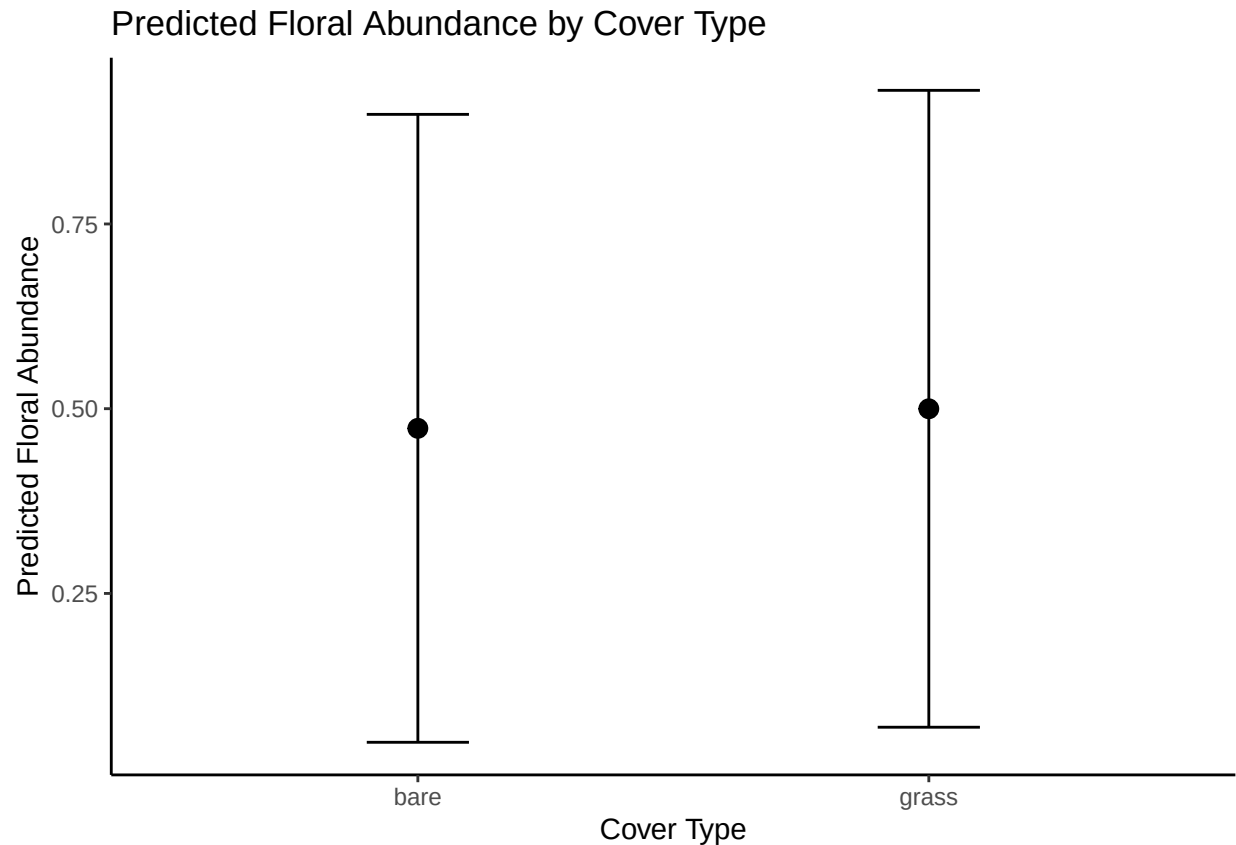
Vegetation

```
# Get predicted response for cover_type, holding other variables at their means
pred_floral_veg <- emmeans(
  floral_noRE,
  ~ cover_type,
  at = list(julian.date_scaled = mean(surveys$julian.date_scaled, na.rm=TRUE)),
  type = "response")
```

```
##
## Re-fitting to get Hessian
```

```
# Convert to dataframe for plotting
pred_floral_veg_df <- as.data.frame(pred_floral_veg)

#plot
ggplot(pred_floral_veg_df, aes(x = cover_type, y = response)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = asymp.LCL, ymax = asymp.UCL), width = 0.2) +
  labs(
    title = "Predicted Floral Abundance by Cover Type",
    x = "Cover Type",
    y = "Predicted Floral Abundance"
  ) +
  theme_classic()
```



Julian date

```
# Define a sequence of values for grass_forb_perc_scaled across its observed range
date_seq <- seq(
  from = min(field_data$julian.date_scaled, na.rm = TRUE),
  to = max(field_data$julian.date_scaled, na.rm = TRUE),
  length.out = 50
)

# Get predicted response for grass_forb_perc_scaled, holding other variables at their means
pred_floral_date <- emmeans(floral_noRE,
  ~ julian.date_scaled,
  at = list(julian.date_scaled = date_seq,
    cover_type = "grass"),
  type = "response")
```

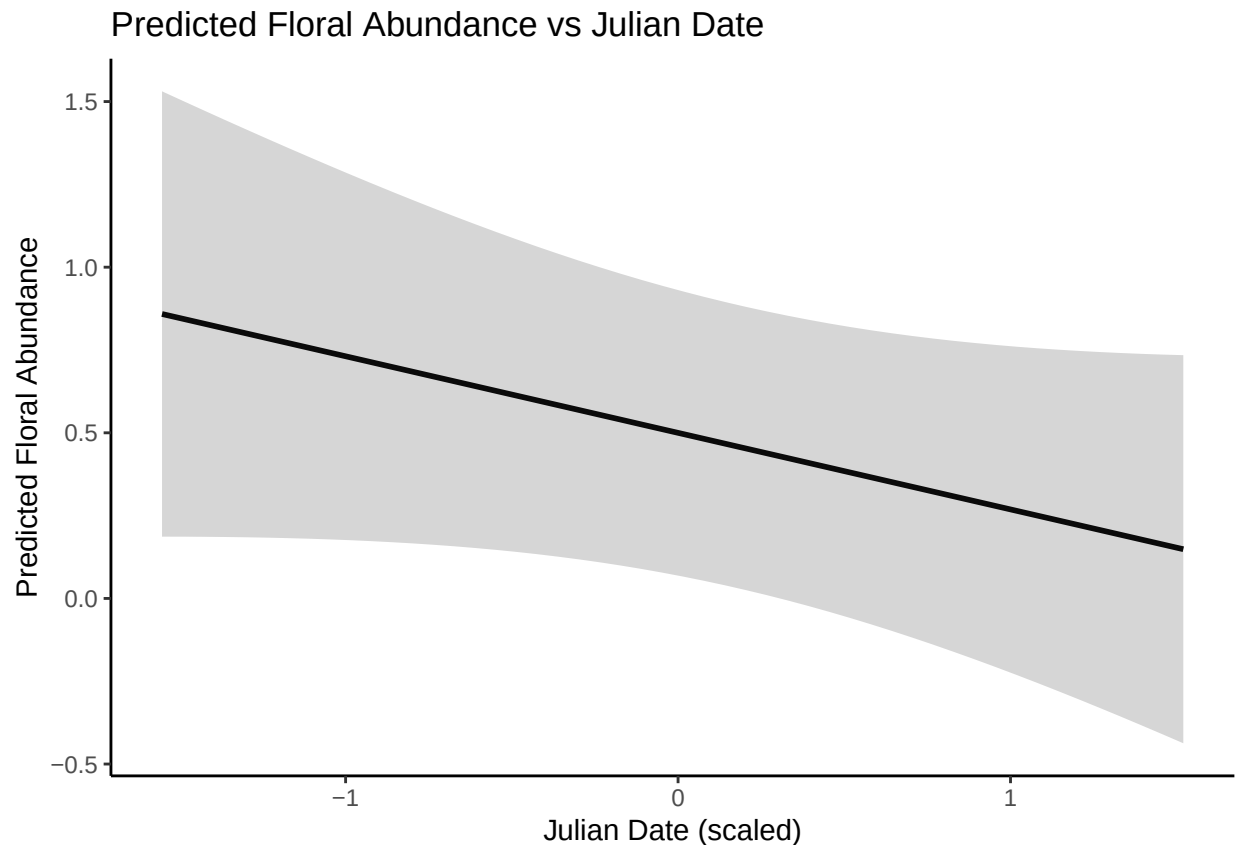
```
##
## Re-fitting to get Hessian
```

```
# Convert to dataframe for plotting
pred_floral_date_df <- as.data.frame(pred_floral_date)

ggplot(pred_floral_date_df, aes(x = julian.date_scaled, y = response)) +
  geom_line(size = 1) +
  geom_ribbon(aes(ymin = asymp.LCL, ymax = asymp.UCL), alpha = 0.2, color = NA) +
  labs(title = "Predicted Floral Abundance vs Julian Date",
```

```
x = "Julian Date (scaled)",
y = "Predicted Floral Abundance") +
theme_classic()
```

```
## Warning: Using 'size' aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use 'linewidth' instead.
## This warning is displayed once every 8 hours.
## Call 'lifecycle::last_lifecycle_warnings()' to see where this warning was
## generated.
```



Findings

No significant effect of either grass+ forb% or julian date.

Number of nest-searching queens vs flowers, rodents, and field management technique

Start by plotting a zero-inflated poisson, since # of nest-searching queens is count data with a lot of zeros. I added bee_season_type as a covariate, since during my exploratory data analyses I saw that there was only a significant different in nest-searching queens among grassy vs bare sites during nesting season.

Steps

1. Try multiple random effects structures

- best fit = no random effect

```
NSQ_full <- glmmTMB(nest_searching_queens ~ cover_type +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type +
  (1|area/site_id/plot_id),
  ziformula = ~ cover_type +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type +
  (1|area/site_id/plot_id),
  family = poisson,
  data = field_data)

NSQ_noArea <- glmmTMB(nest_searching_queens ~ cover_type +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type +
  (1|site_id/plot_id),
  ziformula = ~ cover_type +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type +
  (1|site_id/plot_id),
  family = poisson,
  data = field_data)

NSQ_plotOnly <- glmmTMB(nest_searching_queens ~ cover_type +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type +
  (1|plot_id),
  ziformula = ~ cover_type +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type +
  (1|plot_id),
  family = poisson,
  data = field_data)

NSQ_siteOnly <- glmmTMB(nest_searching_queens ~ cover_type +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type +
  (1|site_id),
  ziformula = ~ cover_type +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
```

```

        bee_season_type +
        (1|site_id),
family = poisson,
data = field_data)

NSQ_AreaPlot <- glmmTMB(nest_searching_queens ~ cover_type +
        bombus_floral_abun_scaled +
        num_burrows_scaled +
        bee_season_type +
        (1|area/plot_id),
ziformula = ~ cover_type +
        bombus_floral_abun_scaled +
        num_burrows_scaled +
        bee_season_type +
        (1|area/plot_id),
family = poisson,
data = field_data)

NSQ_noRE <- glmmTMB(nest_searching_queens ~ cover_type +
        bombus_floral_abun_scaled +
        num_burrows_scaled +
        bee_season_type,
ziformula = ~ cover_type +
        bombus_floral_abun_scaled +
        num_burrows_scaled +
        bee_season_type, # zero-inflation model
family = poisson, #poisson
data = field_data)

#compare models using maximum likelihood test
anova(NSQ_full, NSQ_noArea, NSQ_plotOnly, NSQ_siteOnly, NSQ_AreaPlot, NSQ_noRE)

```

```

## Data: field_data
## Models:
## NSQ_noRE: nest_searching_queens ~ cover_type + bombus_floral_abun_scaled + , zi=~cover_type + bombus
## NSQ_noRE:      num_burrows_scaled + bee_season_type, zi=      bee_season_type, disp=~1
## NSQ_plotOnly: nest_searching_queens ~ cover_type + bombus_floral_abun_scaled + , zi=~cover_type + bomb
## NSQ_plotOnly:      num_burrows_scaled + bee_season_type + (1 | plot_id), zi=      bee_season_type + (1
## NSQ_siteOnly: nest_searching_queens ~ cover_type + bombus_floral_abun_scaled + , zi=~cover_type + bomb
## NSQ_siteOnly:      num_burrows_scaled + bee_season_type + (1 | site_id), zi=      bee_season_type + (1
## NSQ_noArea: nest_searching_queens ~ cover_type + bombus_floral_abun_scaled + , zi=~cover_type + bomb
## NSQ_noArea:      num_burrows_scaled + bee_season_type + (1 | site_id/plot_id), zi=      bee_season_type
## NSQ_AreaPlot: nest_searching_queens ~ cover_type + bombus_floral_abun_scaled + , zi=~cover_type + bomb
## NSQ_AreaPlot:      num_burrows_scaled + bee_season_type + (1 | area/plot_id), zi=      bee_season_type
## NSQ_full: nest_searching_queens ~ cover_type + bombus_floral_abun_scaled + , zi=~cover_type + bombus
## NSQ_full:      num_burrows_scaled + bee_season_type + (1 | area/site_id/plot_id), zi=      bee_season_t
##
##      Df    AIC    BIC  logLik deviance Chisq Chi Df Pr(>Chisq)
## NSQ_noRE      10 141.76 171.18 -60.881   121.76
## NSQ_plotOnly  12 145.76 181.06 -60.881   121.76      0      2      1
## NSQ_siteOnly  12 145.76 181.06 -60.881   121.76      0      0      1
## NSQ_noArea    14 149.76 190.94 -60.881   121.76      0      2      1
## NSQ_AreaPlot  14 149.76 190.94 -60.881   121.76      0      0      1
## NSQ_full      16 153.76 200.83 -60.881   121.76      0      2      1

```

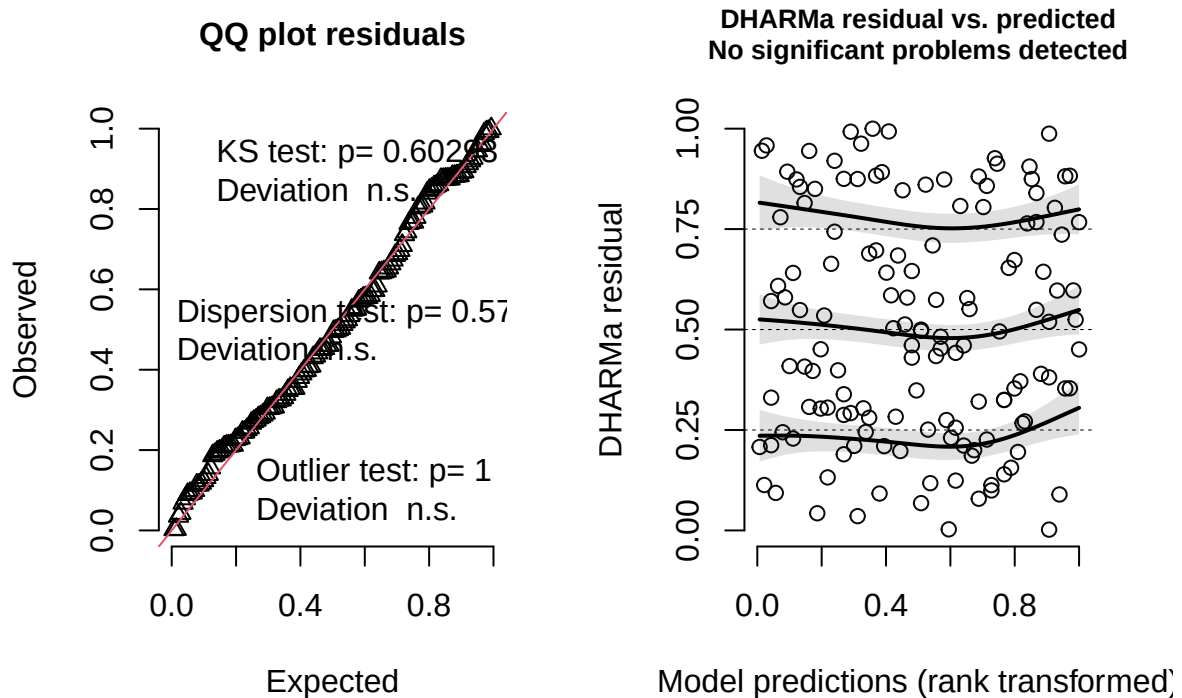
```
##results tells us the model without random effects is best (lowest AIC).
```

2. Check diagnostics of best model.

- No significant problems detected.

```
residuals_nsq_noRE <- simulateResiduals(fittedModel = NSQ_noRE, plot = TRUE)
```

DHARMA residual



```
##looks good
```

3. Look at results

```
summary(NSQ_noRE)
```

```
## Family: poisson ( log )
## Formula:
## nest_searching_queens ~ cover_type + bombus_floral_abun_scaled +
##   num_burrows_scaled + bee_season_type
## Zero inflation:
## ~cover_type + bombus_floral_abun_scaled + num_burrows_scaled +
##   bee_season_type
## Data: field_data
##
```

```
##      AIC      BIC    logLik -2*log(L)  df.resid
##    141.8    171.2    -60.9    121.8      130
##
##
## Conditional model:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      0.31409   0.56344   0.557  0.57721
## cover_typegrass -0.29160   0.59492  -0.490  0.62403
## bombus_floral_abun_scaled 0.08987   0.20929   0.429  0.66763
## num_burrows_scaled  0.83314   0.28331   2.941  0.00327 **
## bee_season_typerworker -0.31290   1.05188  -0.298  0.76611
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Zero-inflation model:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      2.2041    0.8523   2.586  0.00971 **
## cover_typegrass  -4.0585    1.7345  -2.340  0.01929 *
## bombus_floral_abun_scaled -0.4732    0.4522  -1.046  0.29536
## num_burrows_scaled  1.8226    0.7979   2.284  0.02236 *
## bee_season_typerworker  4.2936    1.9276   2.228  0.02592 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Plots

Number of nest-searching queens vs grass/forb

Conditional model:

```
#create new data grid
nsq_data_cover<- expand.grid(
  cover_type = unique(field_data$cover_type),
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled),
  num_burrows_scaled = mean(field_data$num_burrows_scaled),
  bee_season_type = "nestsearching"
)

# Predictions for conditional model (count)
pred_nsq_cover_cond <- predict(NSQ_noRE,
  newdata = nsq_data_cover,
  type = "conditional",
  se.fit = TRUE)

# Predictions for zero-inflated model
pred_nsq_cover_zi <- predict(NSQ_noRE,
  newdata = nsq_data_cover,
  type = "zprob",
  se.fit = TRUE)

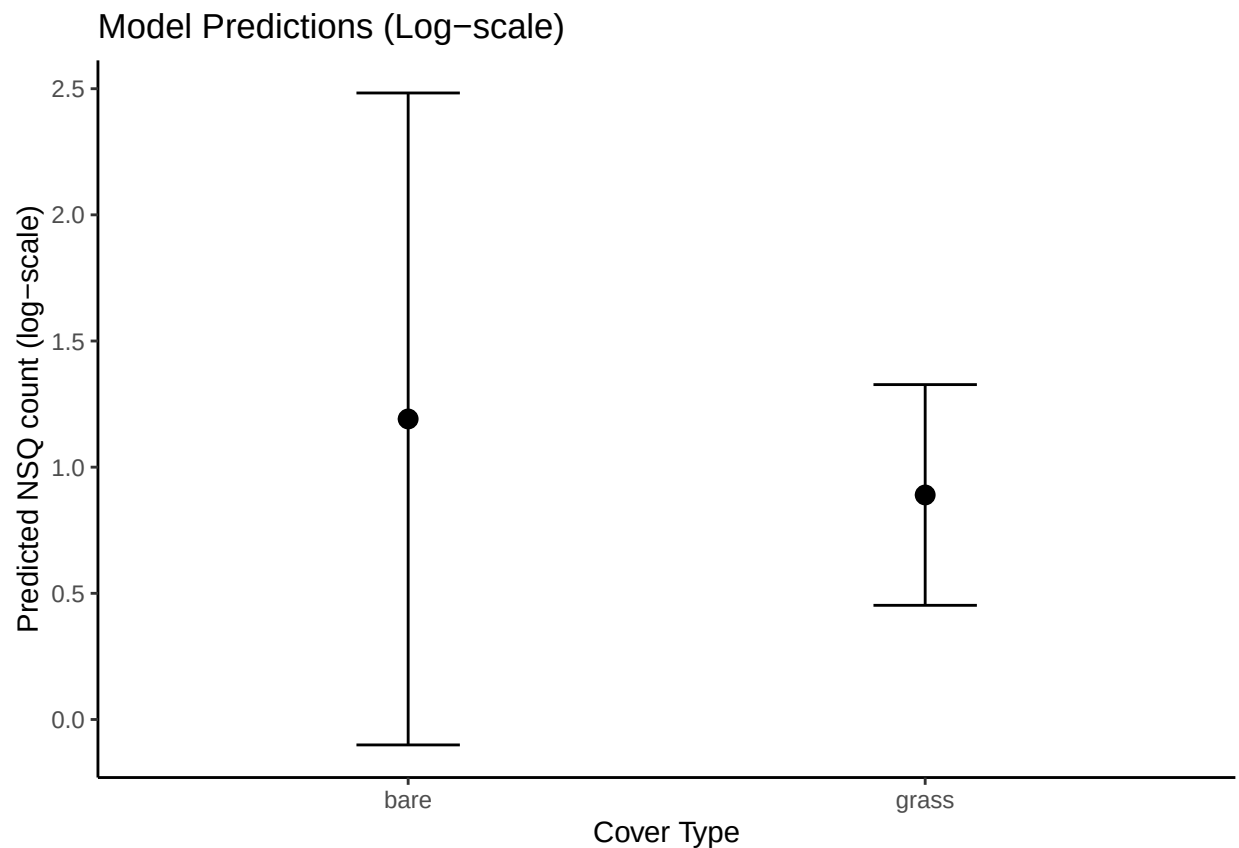
# Combine predictions with data frame
pred_nsq_cover_df <- cbind(nsq_data_cover,
  cond_fit = pred_nsq_cover_cond$fit,
  cond_se = pred_nsq_cover_cond$se.fit,
```

```

        zi_fit = pred_nsq_cover_zi$fit,
        zi_se = pred_nsq_cover_zi$se.fit)

#plot
ggplot(pred_nsq_cover_df, aes(x = cover_type, y = cond_fit)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = cond_fit - 1.96*cond_se, ymax = cond_fit + 1.96*cond_se), width = 0.2, position = position_dodge())
  labs(title = "Model Predictions (Log-scale)",
        x = "Cover Type",
        y = "Predicted NSQ count (log-scale)") +
  theme_classic()

```

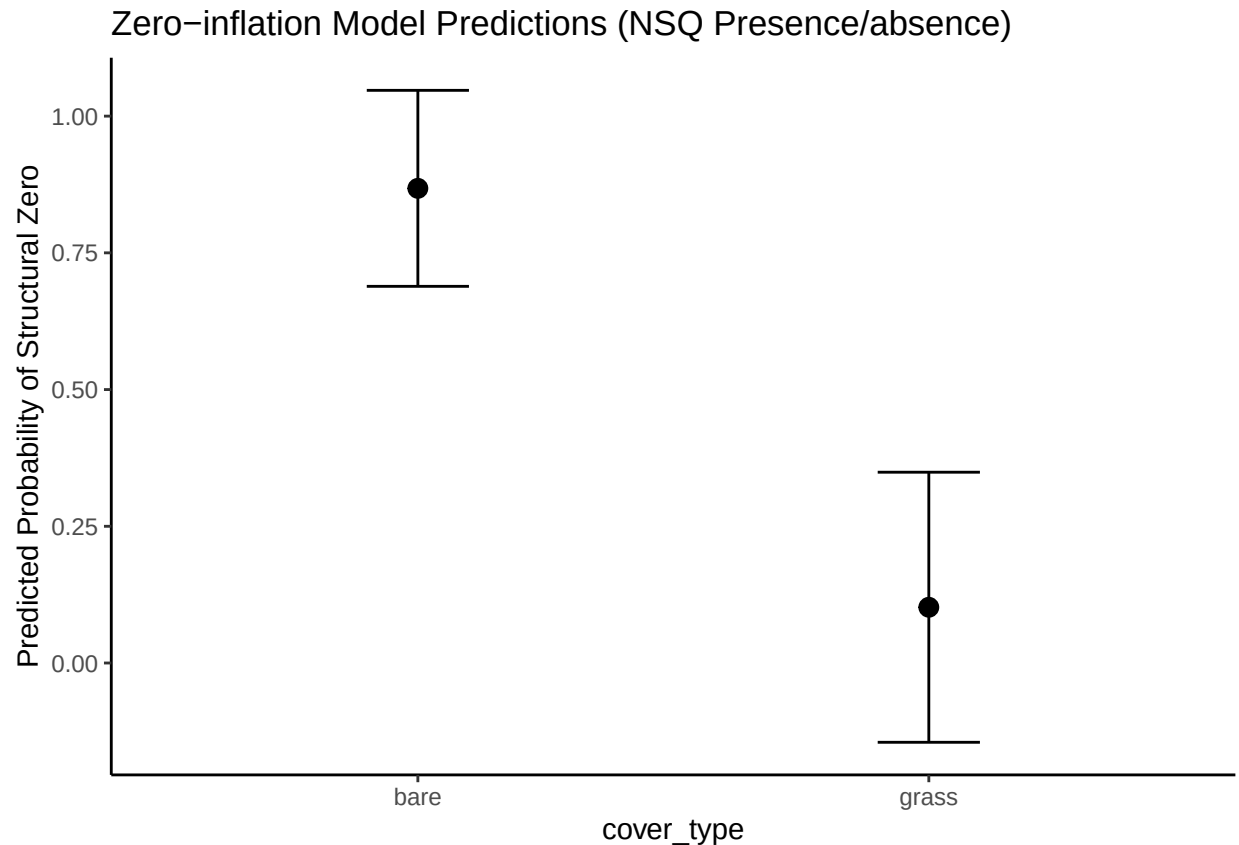


Zero-inflated model:

```

# Plot zi predictions
ggplot(pred_nsq_cover_df, aes(x = cover_type, y = zi_fit)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = zi_fit - 1.96*zi_se, ymax = zi_fit + 1.96*zi_se), width = 0.2, position = position_dodge())
  labs(title = "Zero-inflation Model Predictions (NSQ Presence/absence)", y = "Predicted Probability of")
  theme_classic()

```



Number of nest-searching queens vs floral abundance

Conditional model:

```
# Define a sequence of values for bombus_floral_abun_scaled across its observed range
floral_seq <- seq(
  from = min(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  to = max(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  length.out = 50
)

# Create data grid over bombus_floral_abun_scaled
nsq_data_floral <- data.frame(
  bombus_floral_abun_scaled = floral_seq,
  cover_type = "grass",
  num_burrows_scaled = mean(field_data$num_burrows_scaled),
  bee_season_type = "nestsearching"
)

# Predictions for conditional model (count)
pred_nsq_floral_cond <- predict(NSQ_noRE, newdata = nsq_data_floral, type = "conditional", se.fit = TRUE)

# Predictions for zero-inflation model
pred_nsq_floral_zi <- predict(NSQ_noRE, newdata = nsq_data_floral, type = "zprob", se.fit = TRUE)

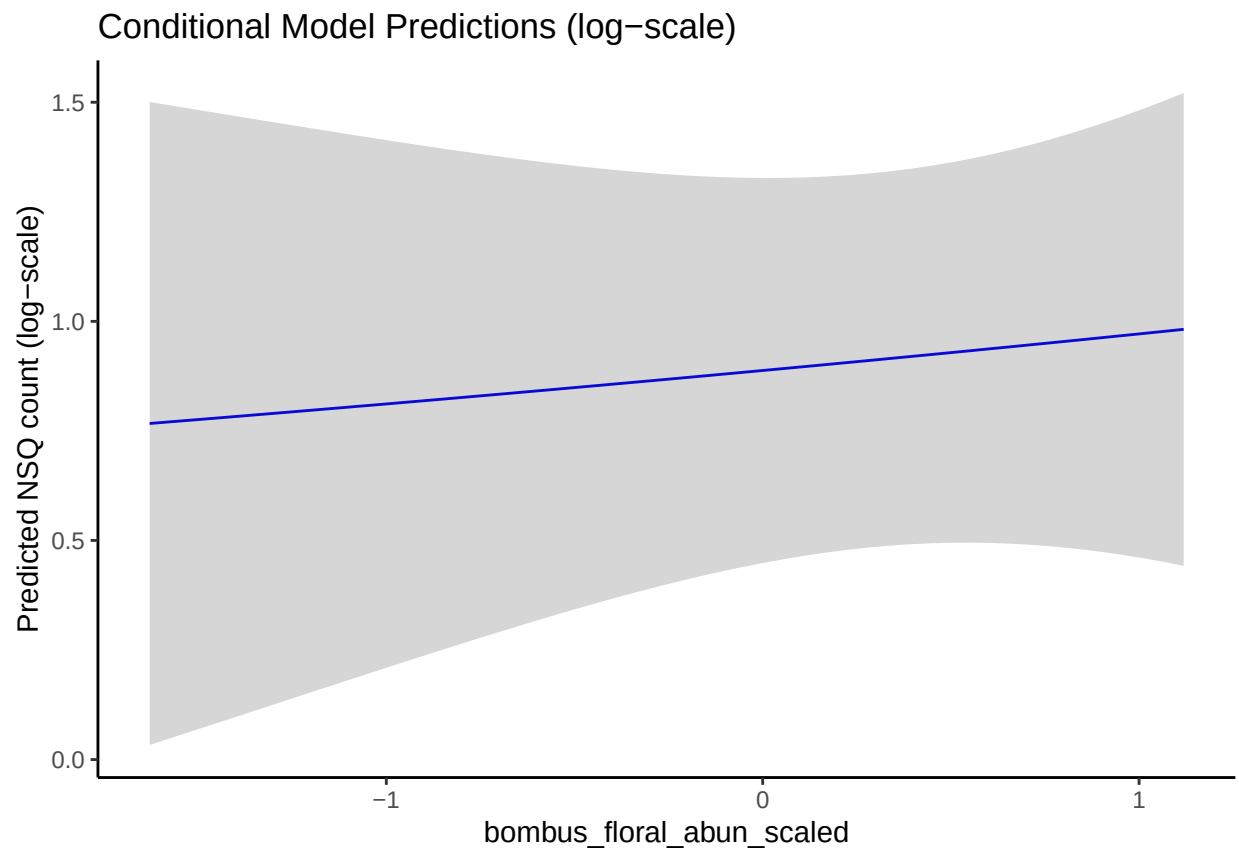
# Combine predictions with newdata
pred_nsq_floral_df <- cbind(nsq_data_floral,
```

```

cond_fit = pred_nsq_floral_cond$fit,
cond_se = pred_nsq_floral_cond$se.fit,
zi_fit = pred_nsq_floral_zi$fit,
zi_se = pred_nsq_floral_zi$se.fit)

# Plot conditional predictions
ggplot(pred_nsq_floral_df, aes(x = bombus_floral_abun_scaled, y = cond_fit)) +
  geom_line(color = "blue") +
  geom_ribbon(aes(ymin = cond_fit - 1.96*cond_se, ymax = cond_fit + 1.96*cond_se), alpha = 0.2) +
  labs(title = "Conditional Model Predictions (log-scale)", y = "Predicted NSQ count (log-scale)") +
  theme_classic()

```

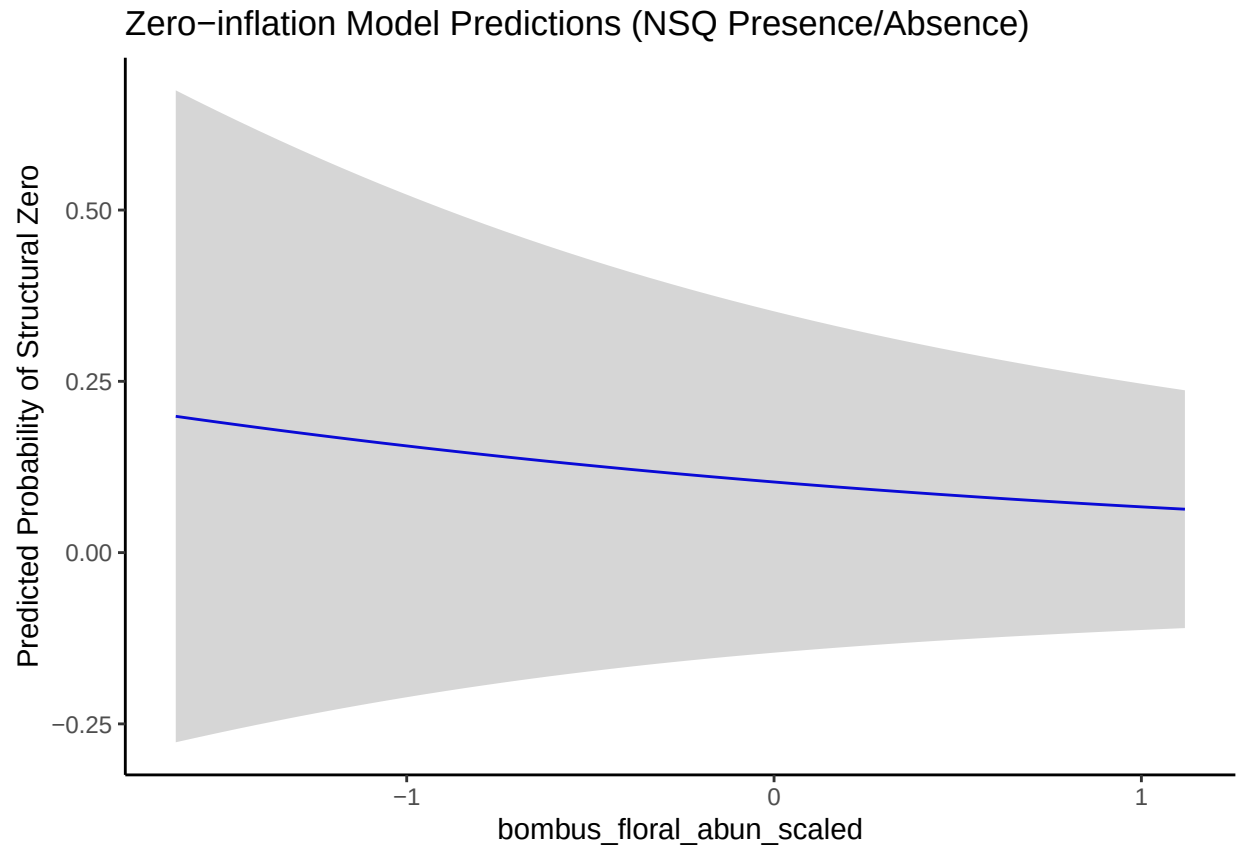


Zero-inflated model:

```

# Plot zi predictions
ggplot(pred_nsq_floral_df, aes(x = bombus_floral_abun_scaled, y = zi_fit)) +
  geom_line(color = "blue") +
  geom_ribbon(aes(ymin = zi_fit - 1.96*zi_se, ymax = zi_fit + 1.96*zi_se), alpha = 0.2) +
  labs(title = "Zero-inflation Model Predictions (NSQ Presence/Absence)", y = "Predicted Probability of") +
  theme_classic()

```



Number of nest-searching queens vs number of rodent burrows

Conditional model:

```
# Create data grid over rodent_burrows_scaled
nsq_data_rodent <- data.frame(
  num_burrows_scaled = seq(min(field_data$num_burrows_scaled), max(field_data$num_burrows_scaled), length.out = 100),
  cover_type = "grass",
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled),
  bee_season_type = "nestsearching"
)

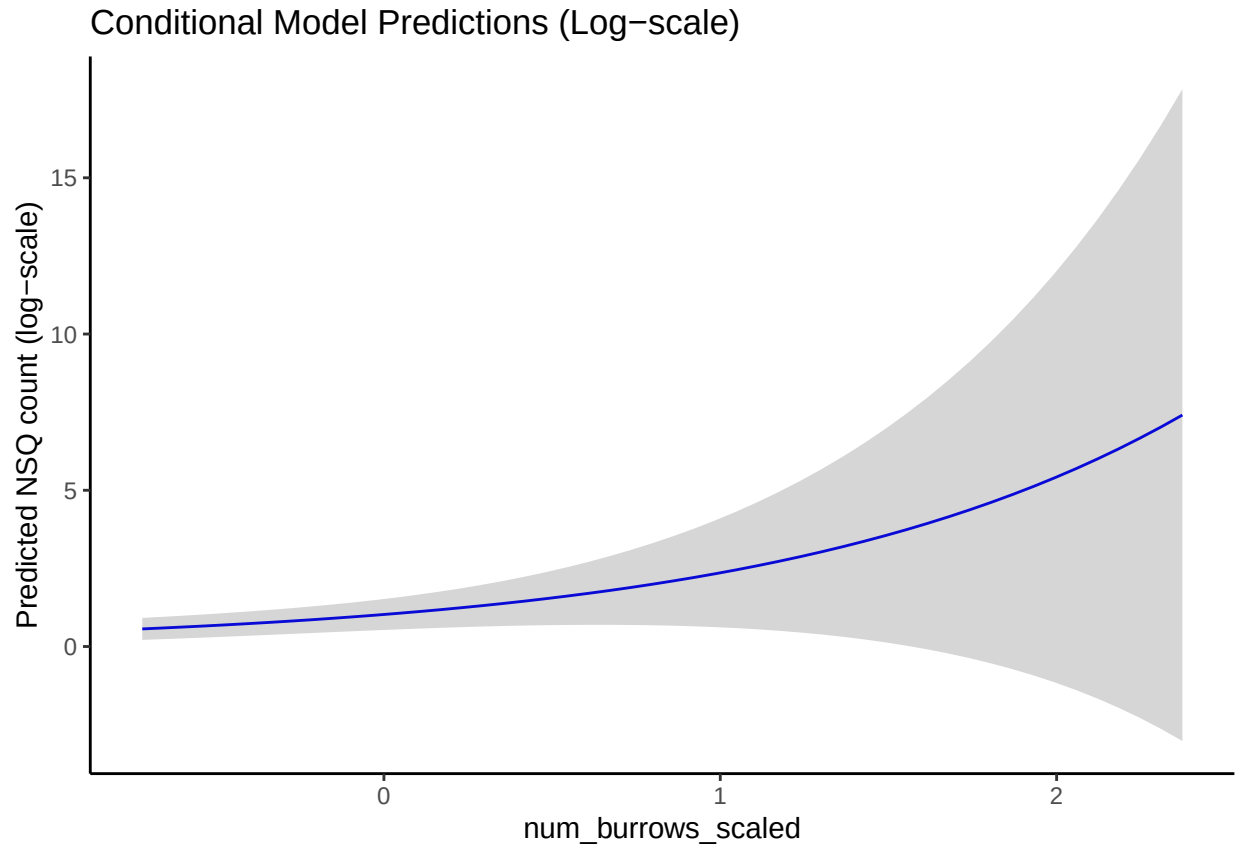
# Predictions for conditional model (count)
pred_nsq_rodent_cond <- predict(NSQ_noRE, newdata = nsq_data_rodent, type = "conditional", se.fit = TRUE)

# Predictions for zero-inflation model
pred_nsq_rodent_zi <- predict(NSQ_noRE, newdata = nsq_data_rodent, type = "zprob", se.fit = TRUE)

# Combine predictions with newdata
pred_nsq_rodent_df <- cbind(nsq_data_rodent,
  cond_fit = pred_nsq_rodent_cond$fit,
  cond_se = pred_nsq_rodent_cond$se.fit,
  zi_fit = pred_nsq_rodent_zi$fit,
  zi_se = pred_nsq_rodent_zi$se.fit)

# Plot conditional predictions
ggplot(pred_nsq_rodent_df, aes(x = num_burrows_scaled, y = cond_fit)) +
```

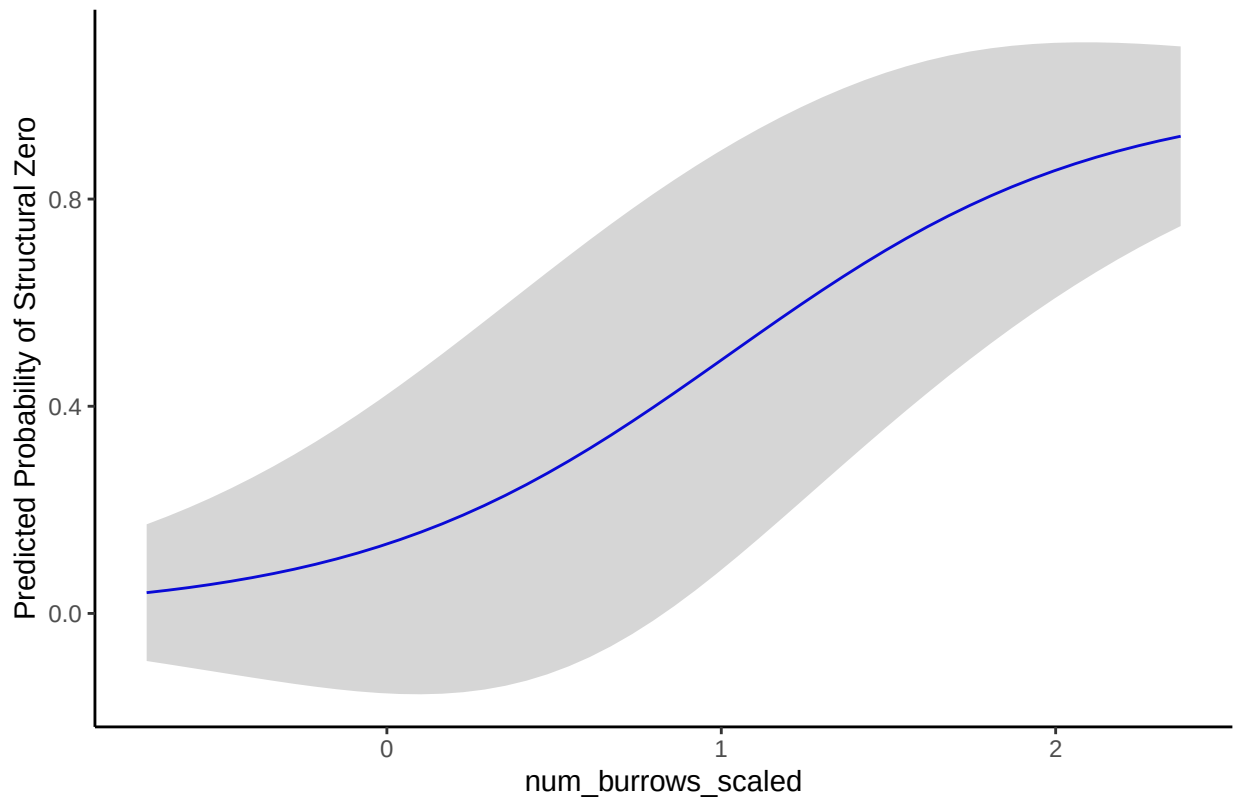
```
geom_line(color = "blue") +
geom_ribbon(aes(ymin = cond_fit - 1.96*cond_se, ymax = cond_fit + 1.96*cond_se), alpha = 0.2) +
labs(title = "Conditional Model Predictions (Log-scale)", y = "Predicted NSQ count (log-scale)") +
theme_classic()
```



Zero-inflated model:

```
ggplot(pred_nsq_rodent_df, aes(x = num_burrows_scaled, y = zi_fit)) +
geom_line(color = "blue") +
geom_ribbon(aes(ymin = zi_fit - 1.96*zi_se, ymax = zi_fit + 1.96*zi_se), alpha = 0.2) +
labs(title = "Zero-inflation Model Predictions (NSQ Presence/Absence)", y = "Predicted Probability of") +
theme_classic()
```

Zero-inflation Model Predictions (NSQ Presence/Absence)



Number of nest-searching queens vs season type

Conditional model:

```
# Create newdata with discrete factor levels for bee_season_type
nsq_data_season <- data.frame(
  cover_type = "grass",
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled),
  num_burrows_scaled = mean(field_data$num_burrows_scaled),
  bee_season_type = factor(c("nestsearching", "worker"), levels = levels(field_data$bee_season_type))
)

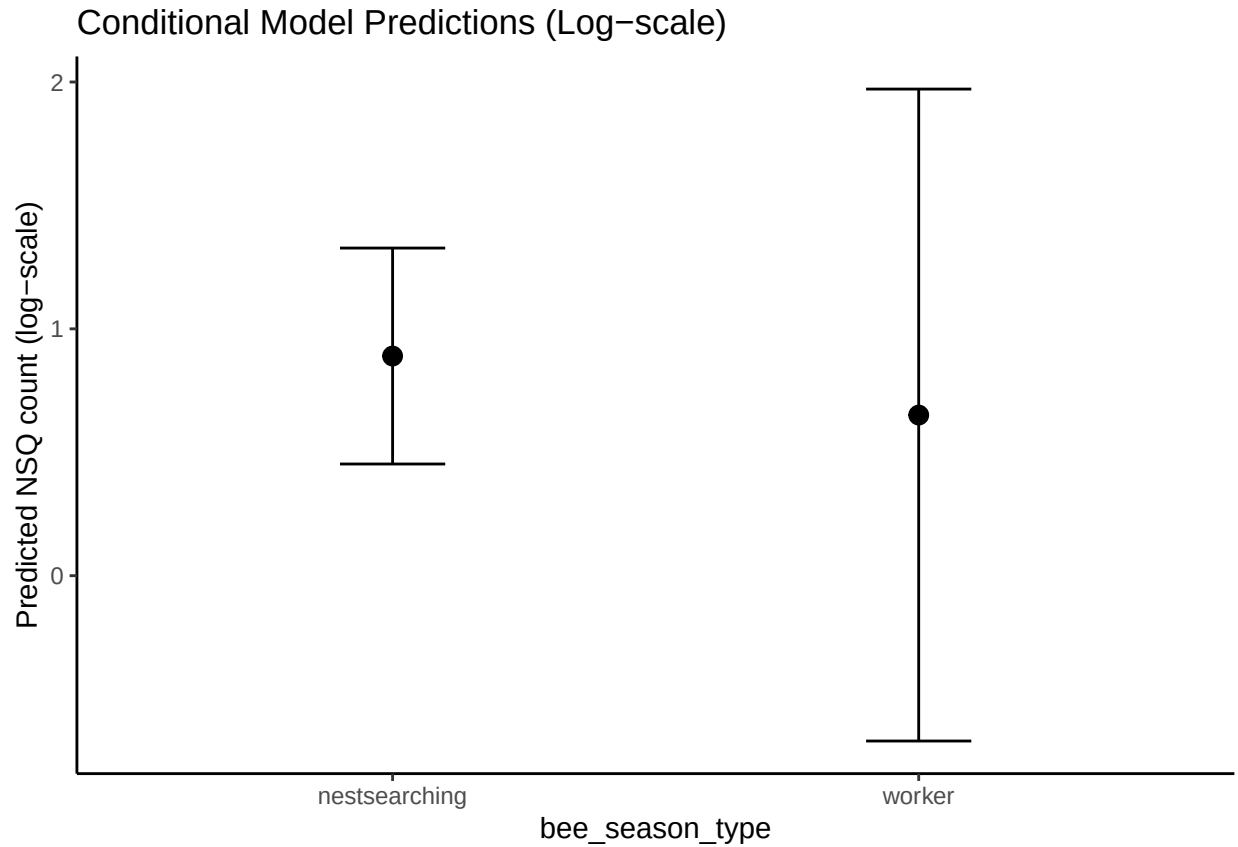
# Predictions for conditional model (count)
pred_nsq_season_cond <- predict(NSQ_noRE, newdata = nsq_data_season, type = "conditional", se.fit = TRUE)

# Predictions for zero-inflation model
pred_nsq_season_zi <- predict(NSQ_noRE, newdata = nsq_data_season, type = "zprob", se.fit = TRUE)

# Combine predictions with newdata
pred_nsq_season_df <- cbind(nsq_data_season,
  cond_fit = pred_nsq_season_cond$fit,
  cond_se = pred_nsq_season_cond$se.fit,
  zi_fit = pred_nsq_season_zi$fit,
  zi_se = pred_nsq_season_zi$se.fit)

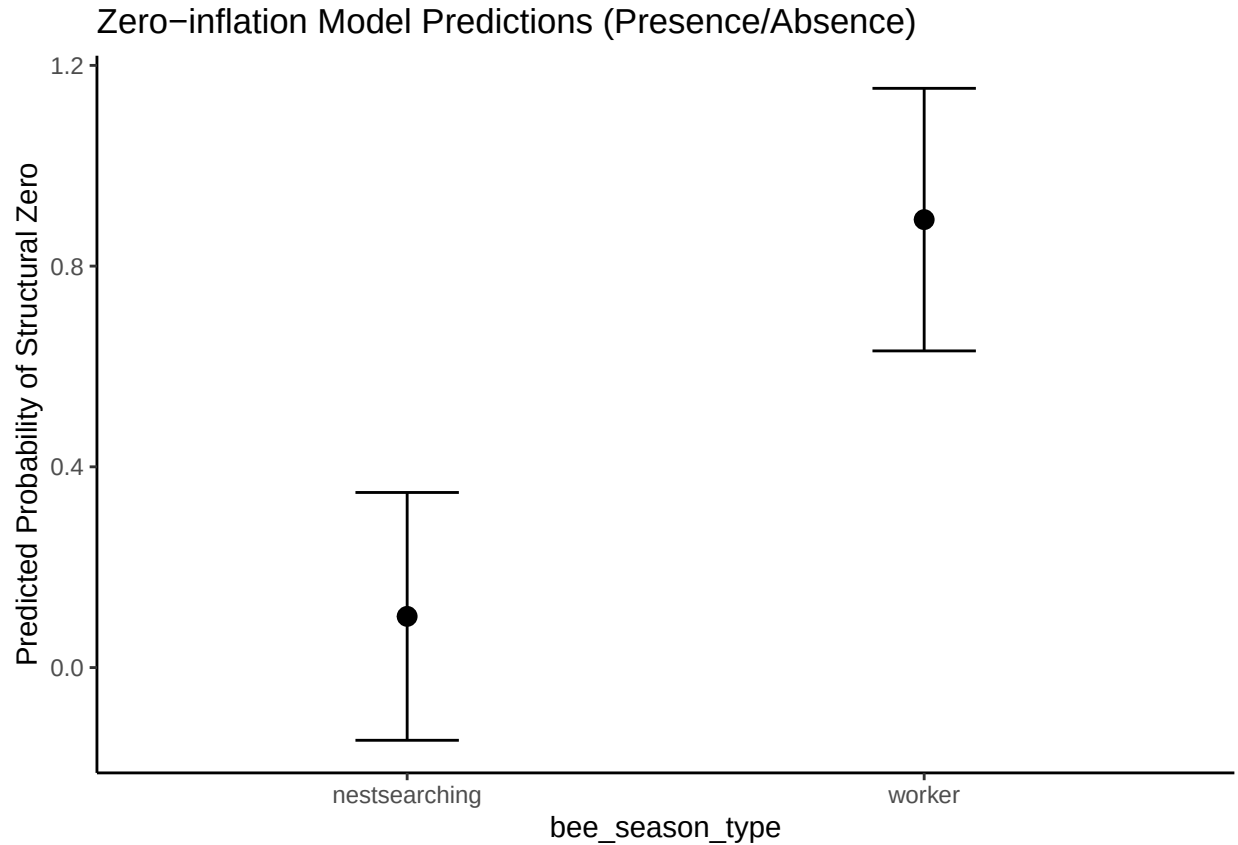
# Plot conditional predictions
ggplot(pred_nsq_season_df, aes(x = bee_season_type, y = cond_fit)) +
```

```
geom_point(size=3) +
geom_errorbar(aes(ymin = cond_fit - 1.96*cond_se, ymax = cond_fit + 1.96*cond_se), width = 0.2, position = position_dodge(0.5)) +
labs(title = "Conditional Model Predictions (Log-scale)", y = "Predicted NSQ count (log-scale)") +
theme_classic()
```



Zero-inflated model:

```
ggplot(pred_nsq_season_df, aes(x = bee_season_type, y = zi_fit)) +
geom_point(size=3) +
geom_errorbar(aes(ymin = zi_fit - 1.96*zi_se, ymax = zi_fit + 1.96*zi_se), width = 0.2, position = position_dodge(0.5)) +
labs(title = "Zero-inflation Model Predictions (Presence/Absence)", y = "Predicted Probability of Str") +
theme_classic()
```



Findings

In conditional model (modelling abundance of nest-searching queens):

- no significant effect of grass+forb%, floral abundance, bee season
- significant positive effect of number of burrows

In zero-inflation model (modelling presence/absence of nest-searching queens):

- no significant effect of floral abundance
- Significant effect of grass+forb% (less likely to be absent with more grass/forb), number of burrows (more likely to be absent with more burrows), bee season (more likely to be absent during worker season than nest-searching season).

Interestingly, the number of rodent burrows is associated with more bombus nest-searching in the conditional model, but the number of rodent burrows is associated with more structural zeros in the zero-inflation model. This means that if a site is suitable for nesting, more rodent burrows is associated with more bombus nest-searching. However, in some cases, rodent burrows may signal inhospitable habitat conditions for bumble bees.

Number of workers vs nest-searching queens, floral abundance, rodent burrows.

Go through similar model selection steps as for the rodent burrow model.

Steps

1) Check different random effects structures

- best fit = 1|site_id/plot_id -> **area not included**

```
workers_full <- glmmTMB(workers ~ nest_searching_queens_scaled +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type +
  (1|area/site_id/plot_id),
  family = poisson,
  data = field_data
)

workers_noArea <- glmmTMB(workers ~ nest_searching_queens_scaled +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type+
  (1|site_id/plot_id),
  family = poisson,
  data = field_data
)

workers_plotOnly <- glmmTMB(workers ~ nest_searching_queens_scaled +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type+
  (1|plot_id),
  family = poisson,
  data = field_data
)

workers_siteOnly <- glmmTMB(workers ~ nest_searching_queens_scaled +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type+
  (1|site_id),
  family = poisson,
  data = field_data
)

workers_AreaPlot <- glmmTMB(workers ~ nest_searching_queens_scaled +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type+
```

```

      (1|area/plot_id),
      family = poisson,
      data = field_data
    )

workers_noRE <- glmmTMB(workers ~ nest_searching_queens_scaled +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type,
  family = poisson,
  data = field_data
)

#compare models using maximum likelihood test
anova(workers_full, workers_noArea, workers_plotOnly, workers_siteOnly, workers_AreaPlot, workers_noRE)

## Data: field_data
## Models:
## workers_noRE: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=~1
## workers_noRE:      num_burrows_scaled + bee_season_type, zi=~0, disp=~1
## workers_plotOnly: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=~1
## workers_plotOnly:      num_burrows_scaled + bee_season_type + (1 | plot_id), zi=~0, disp=~1
## workers_siteOnly: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=~1
## workers_siteOnly:      num_burrows_scaled + bee_season_type + (1 | site_id), zi=~0, disp=~1
## workers_noArea: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=~1
## workers_noArea:      num_burrows_scaled + bee_season_type + (1 | site_id/plot_id), zi=~0, disp=~1
## workers_AreaPlot: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=~1
## workers_AreaPlot:      num_burrows_scaled + bee_season_type + (1 | area/plot_id), zi=~0, disp=~1
## workers_full: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=~1
## workers_full:      num_burrows_scaled + bee_season_type + (1 | area/site_id/plot_id), zi=~0, disp=~1
##
##      Df      AIC      BIC logLik deviance  Chisq Chi Df Pr(>Chisq)
## workers_noRE      5 197.53 212.24 -93.767  187.53
## workers_plotOnly  6 196.94 214.59 -92.472  184.94 2.5891      1  0.10760
## workers_siteOnly  6 188.19 205.84 -88.097  176.19 8.7494      0  < 2e-16 ***
## workers_noArea    7 185.25 205.85 -85.627  171.25 4.9402      1  0.02624 *
## workers_AreaPlot  7 190.17 210.76 -88.085  176.17 0.0000      0  1.00000
## workers_full      8 187.25 210.79 -85.627  171.25 4.9152      1  0.02662 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

```

##results tells us the model with no area is best (lowest AIC).

```

2. Check diagnostics of best model.

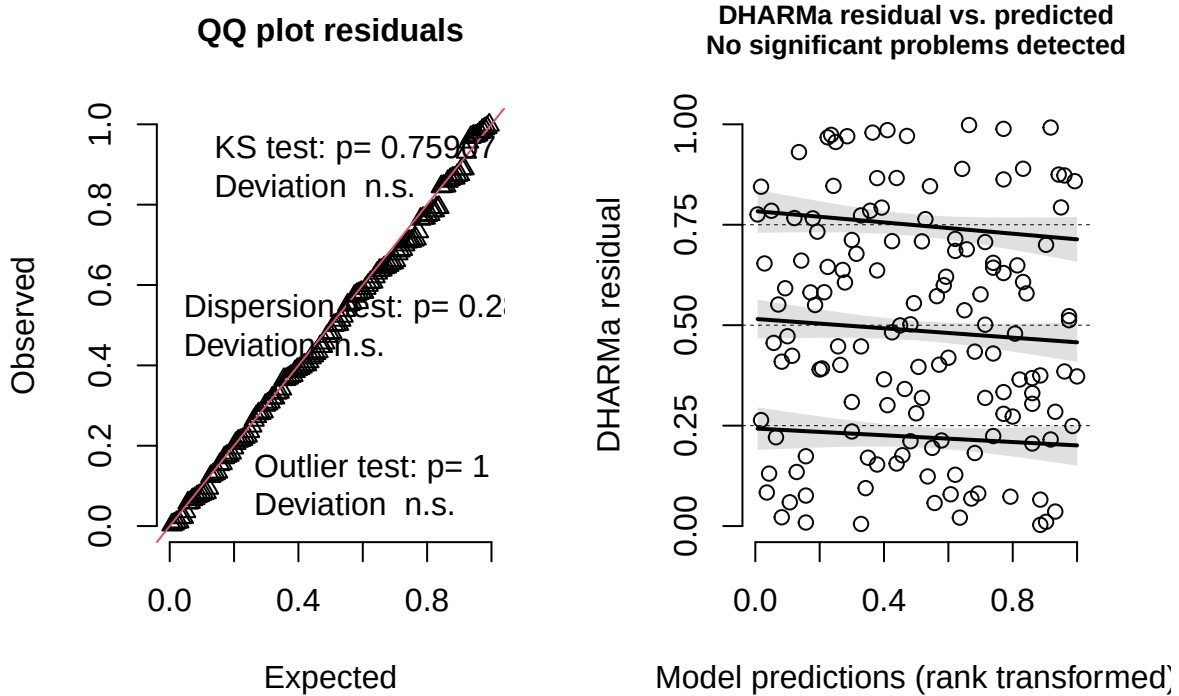
- Residual plots show that there are no significant issues with model fit. Check to see if negative binomial straightens lines in residual plot.

```

#simulate residuals
residuals_workers_noArea <- simulateResiduals(fittedModel = workers_noArea, plot = TRUE)

```

DHARMa residual



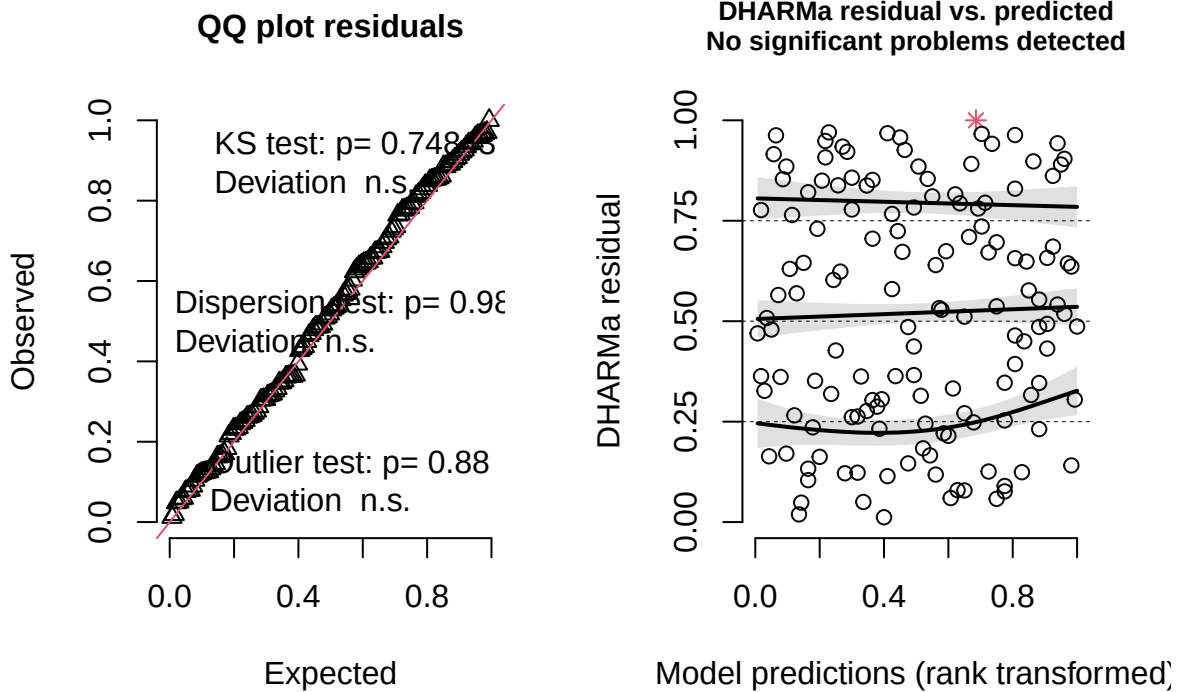
3. Plot negative binomial and check diagnostics.

- Residual plot shows better fit (straighter horizontal lines). AIC is also lower for negative binomial. This shows that the negative binomial is a better fit.

```
worker_negbin <- glmmTMB(workers ~ nest_searching_queens_scaled +
  bombus_floral_abun_scaled +
  num_burrows_scaled +
  bee_season_type +
  (1|site_id/plot_id),
  family = nbinom2,
  data = field_data
)

residuals_workers_negbin <- simulateResiduals(fittedModel = worker_negbin, plot = TRUE)
```

DHARMa residual



```
anova(workers_noArea, worker_negbin)
```

```
## Data: field_data
## Models:
## workers_noArea: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=~
## workers_noArea:      num_burrows_scaled + bee_season_type + (1 | site_id/plot_id), zi=~0, disp=~1
## worker_negbin: workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled + , zi=~0, disp=~1
## worker_negbin:      num_burrows_scaled + bee_season_type + (1 | site_id/plot_id), zi=~0, disp=~1
##           Df    AIC    BIC logLik deviance Chisq Chi Df Pr(>Chisq)
## workers_noArea  7 185.25 205.85 -85.627   171.25
## worker_negbin   8 161.67 185.20 -72.835   145.67 25.584      1 4.235e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
##negative binomial is better (lower AIC)
```

4. Check results of final model

```
summary(worker_negbin)
```

```
## Family: nbinom2 ( log )
## Formula:
## workers ~ nest_searching_queens_scaled + bombus_floral_abun_scaled +
##      num_burrows_scaled + bee_season_type + (1 | site_id/plot_id)
```

```
## Data: field_data
##
##      AIC      BIC    logLik -2*log(L)  df.resid
##    161.7    185.2    -72.8    145.7      132
##
## Random effects:
##
## Conditional model:
## Groups      Name      Variance Std.Dev.
## plot_id:site_id (Intercept) 6.772e-12 2.602e-06
## site_id      (Intercept) 4.216e-09 6.493e-05
## Number of obs: 140, groups: plot_id:site_id, 24; site_id, 12
##
## Dispersion parameter for nbinom2 family (): 0.182
##
## Conditional model:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)      -2.72109    0.56338  -4.830 1.37e-06 ***
## nest_searching_queens_scaled  0.07112    0.55093   0.129  0.8973
## bombus_floral_abun_scaled    0.67688    0.35602   1.901  0.0573 .
## num_burrows_scaled          0.08892    0.31759   0.280  0.7795
## bee_season_typeworker        1.79565    0.71167   2.523  0.0116 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Plots

Number of workers vs number of nest-searching queens

```
# Define a sequence of values for nsq across its observed range
queen_seq <- seq(
  from = min(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  to = max(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  length.out = 50
)

#create new data grid
worker_data_nsq<- expand.grid(
  nest_searching_queens_scaled = queen_seq,
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  bee_season_type = "worker"
)

# Get predicted probabilities and standard errors
pred_worker_nsq <- predict(worker_negbin,
  newdata = worker_data_nsq,
  se.fit = TRUE,
  re.form = NA)

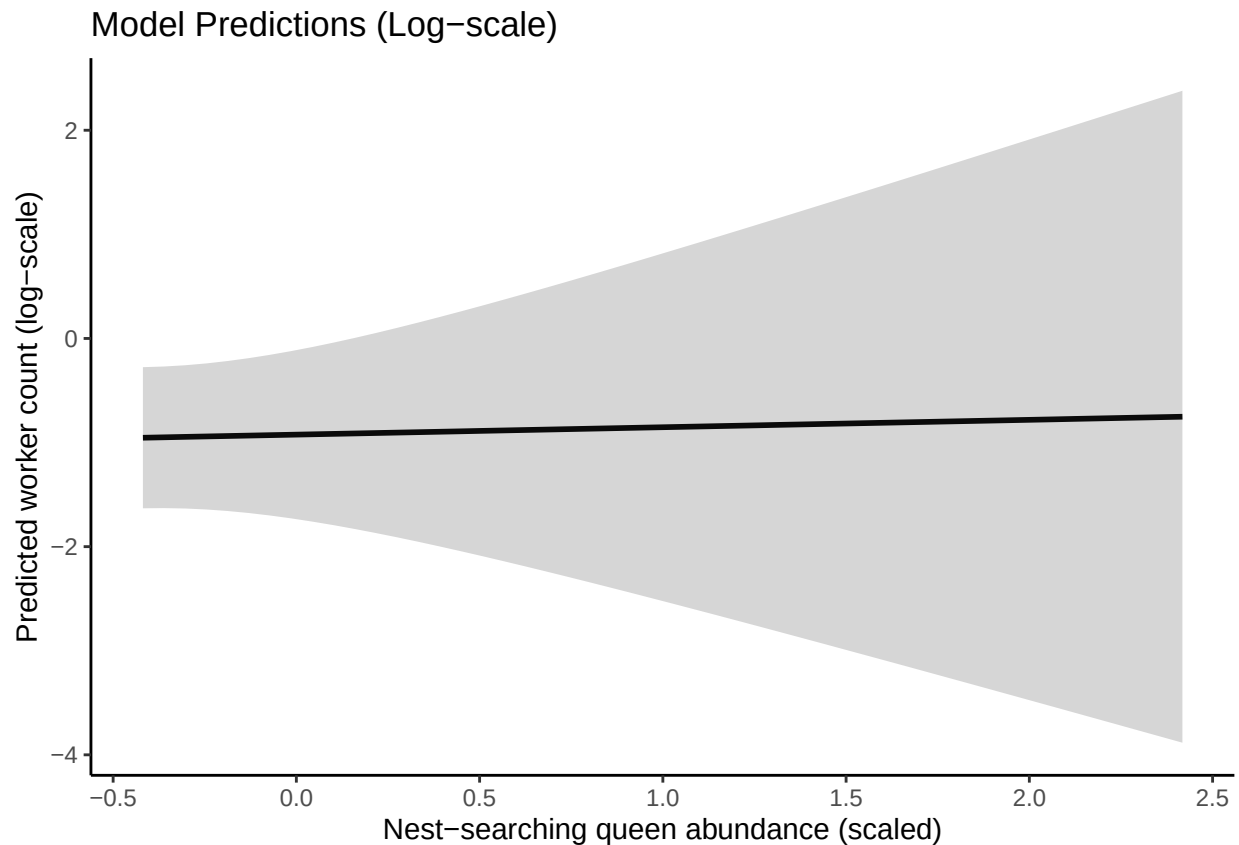
# Combine predictions with data frame
pred_worker_nsq_df <- cbind(worker_data_nsq,
  fit = pred_worker_nsq$fit,
```

```

se = pred_worker_nsq$se.fit)

ggplot(pred_worker_nsq_df, aes(x = nest_searching_queens_scaled, y = fit)) +
  geom_line(size = 1) +
  geom_ribbon(aes(ymin = fit - 1.96*se, ymax = fit + 1.96*se), alpha = 0.2) +
  labs(title = "Model Predictions (Log-scale)",
       x = "Nest-searching queen abundance (scaled)",
       y = "Predicted worker count (log-scale)") +
  theme_classic()

```



Number of workers vs floral abundance

```

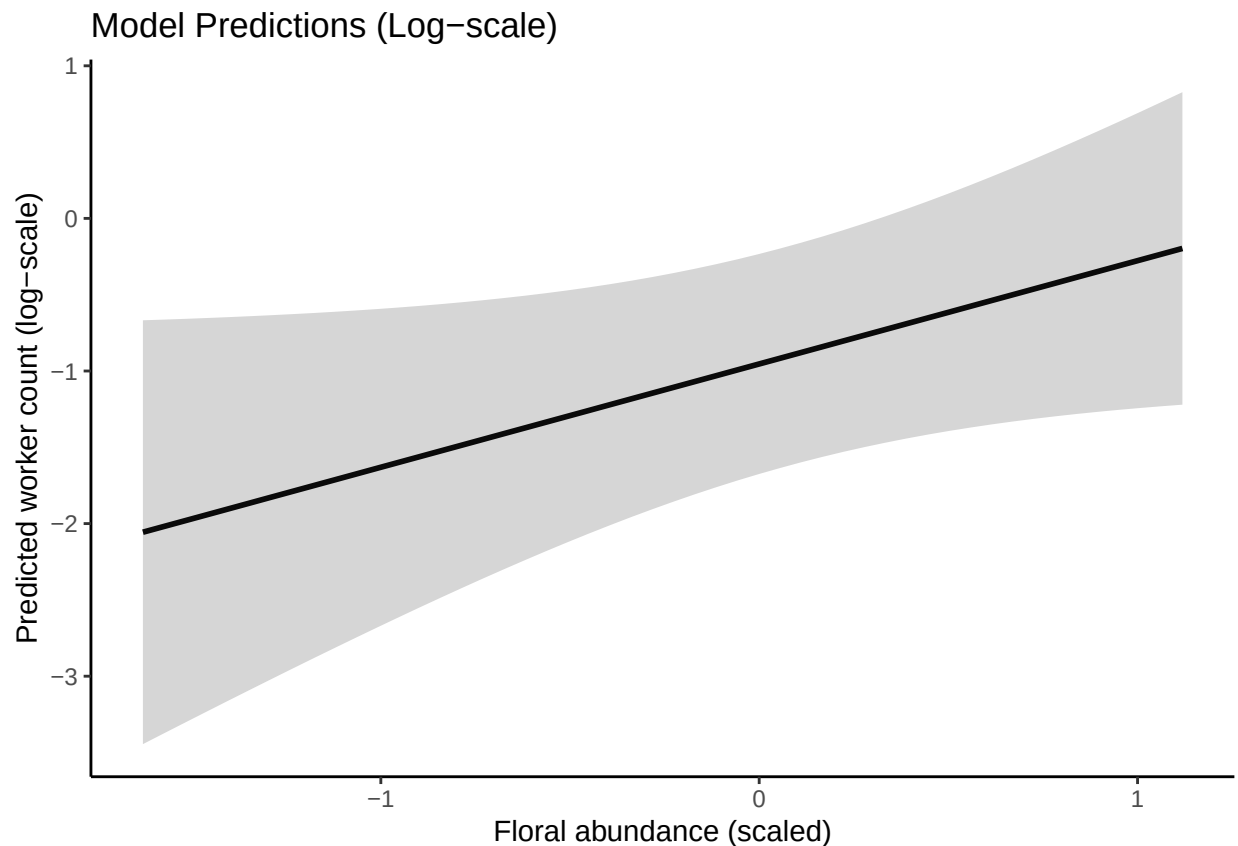
#create new data grid
worker_data_floral<- expand.grid(
  nest_searching_queens_scaled = mean(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  bombus_floral_abun_scaled = floral_seq,
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  bee_season_type = "worker"
)

# Get predicted probabilities and standard errors
pred_worker_floral <- predict(worker_negbin,
                             newdata = worker_data_floral,
                             se.fit = TRUE,
                             re.form = NA)

```

```
# Combine predictions with data frame
pred_worker_floral_df <- cbind(worker_data_floral,
                               fit = pred_worker_floral$fit,
                               se = pred_worker_floral$se.fit)

ggplot(pred_worker_floral_df, aes(x = bombus_floral_abun_scaled, y = fit)) +
  geom_line(size = 1) +
  geom_ribbon(aes(ymin = fit - 1.96*se, ymax = fit + 1.96*se), alpha = 0.2) +
  labs(title = "Model Predictions (Log-scale)",
       x = "Floral abundance (scaled)",
       y = "Predicted worker count (log-scale)") +
  theme_classic()
```



Number of workers vs number of rodent burrows

```
# Define a sequence of values for rodent burrows across its observed range
rodent_seq <- seq(
  from = min(field_data$num_burrows_scaled, na.rm = TRUE),
  to = max(field_data$num_burrows_scaled, na.rm = TRUE),
  length.out = 50)

#create new data grid
worker_data_rodent <- expand_grid(
  nest_searching_queens_scaled = mean(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled = rodent_seq,
```

```

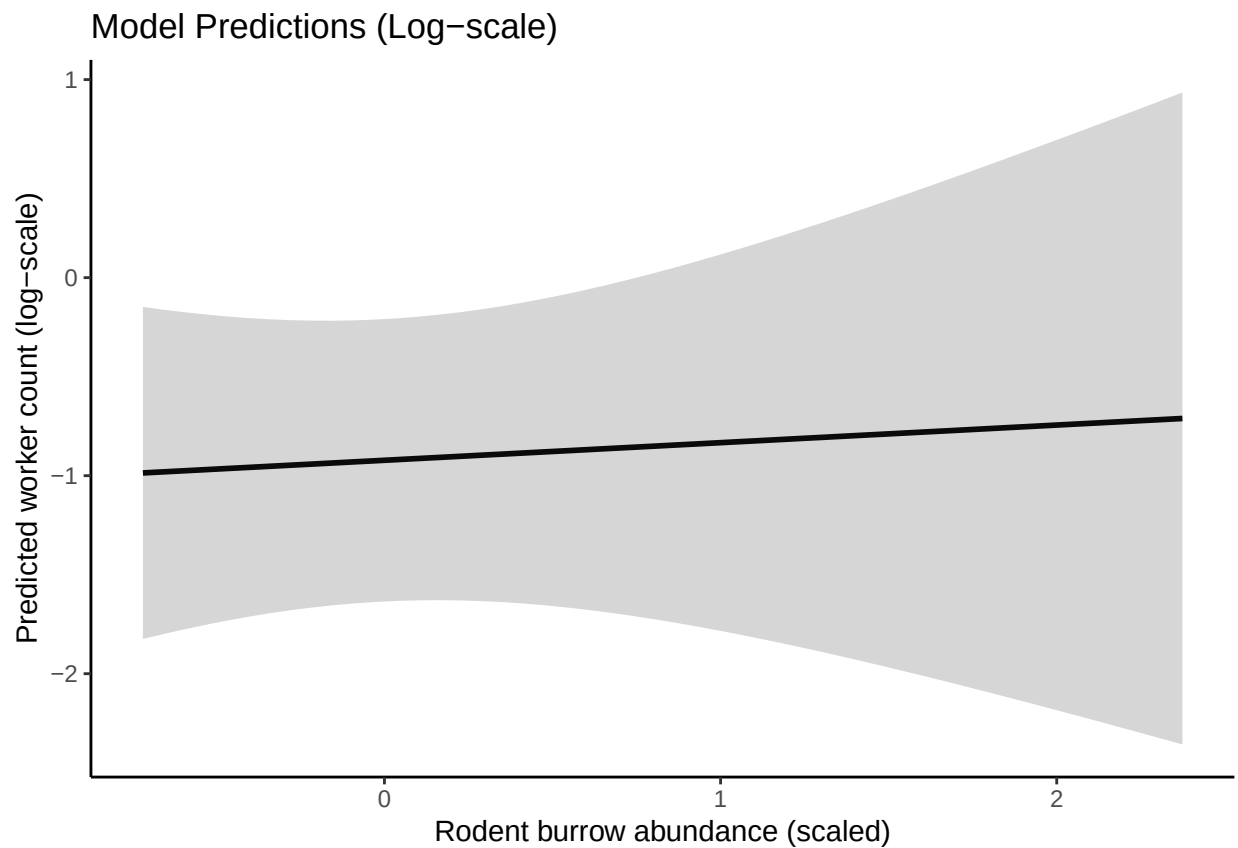
bee_season_type = "worker"
)

# Get predicted probabilities and standard errors
pred_worker_rodent <- predict(worker_negbin,
                              newdata = worker_data_rodent,
                              se.fit = TRUE,
                              re.form = NA)

# Combine predictions with data frame
pred_worker_rodent_df <- cbind(worker_data_rodent,
                               fit = pred_worker_rodent$fit,
                               se = pred_worker_rodent$se.fit)

ggplot(pred_worker_rodent_df, aes(x = num_burrows_scaled, y = fit)) +
  geom_line(size = 1) +
  geom_ribbon(aes(ymin = fit - 1.96*se, ymax = fit + 1.96*se), alpha = 0.2) +
  labs(title = "Model Predictions (Log-scale)",
       x = "Rodent burrow abundance (scaled)",
       y = "Predicted worker count (log-scale)") +
  theme_classic()

```



Number of workers vs season

```

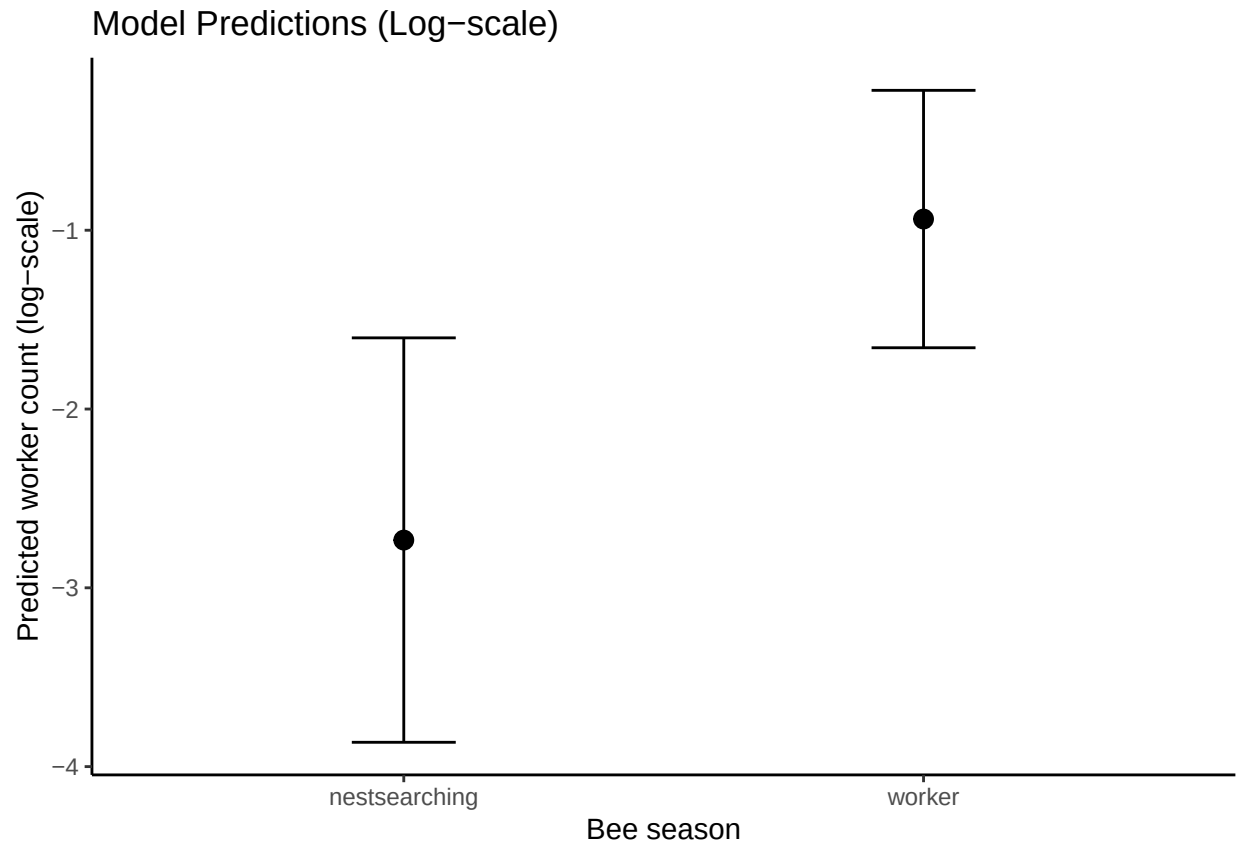
#create new data grid
worker_data_season<- expand.grid(
  nest_searching_queens_scaled = mean(field_data$nest_searching_queens_scaled, na.rm = TRUE),
  bombus_floral_abun_scaled = mean(field_data$bombus_floral_abun_scaled, na.rm = TRUE),
  num_burrows_scaled = mean(field_data$num_burrows_scaled, na.rm = TRUE),
  bee_season_type = unique(field_data$bee_season_type)
)

# Get predicted probabilities and standard errors
pred_worker_season <- predict(worker_negbin,
                              newdata = worker_data_season,
                              se.fit = TRUE,
                              re.form = NA)

# Combine predictions with data frame
pred_worker_season_df <- cbind(worker_data_season,
                               fit = pred_worker_season$fit,
                               se = pred_worker_season$se.fit)

ggplot(pred_worker_season_df, aes(x = bee_season_type, y = fit)) +
  geom_point(size = 3) +
  geom_errorbar(aes(ymin = fit - 1.96*se, ymax = fit + 1.96*se),
               width = 0.2, position = position_dodge(0.6)) +
  labs(title = "Model Predictions (Log-scale)",
       x = "Bee season",
       y = "Predicted worker count (log-scale)") +
  theme_classic()

```



Findings

Number of nest-searching queens and number of rodent burrows is does not significantly effect number of workers. Significantly more workers during worker season than during nest-searching season. Marginally insignificant ($p=0.0573$) positive effect of bombus-relevant floral abundance on worker abundance.

Final Path Diagram

